МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний аерокосмічний університет
«Харківський авіаційний інститут»

Клименко Т.А., Ю. С. Манжос

ТЕСТУВАННЯ ТА ВЕРИФІКАЦІЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ (Курсовий проєкт)

Навчальний посібник

Харків «ХАІ» 2026

УДК 004.65(076.5)
Ш37

Рецензенти:

Клименко, Т.А. та Манжос, Ю. С.
К37 Тестування та верифікація програмного забезпечення (курсовий проєкт) [Електронний ресурс] : навч. посіб. / Ю. С. Манжос – Харків : Нац. аерокосм. ун-т «Харків. авіац. ін-т», 2026. – 90 с.

Наведено завдання до практичних робіт, які дають змогу виконати курсовий проєкт, що є частиною бакалаврської кваліфікаційної роботи.
Для студентів усіх спеціальностей галузі знань 12 «Інформаційні технології» денної і дистанційної форм навчання.

Бібліогр.: 6 назв

УДК 004.65(076.5)

© Манжос Ю. С. 2026
© Національний аерокосмічний університет «Харківський авіаційний інститут», 2026

ПЕРЕДМОВА

Посібник до курсового проєкту з дисципліни **«Тестування та верифікація програмного забезпечення»** розроблено з метою методичної підтримки здобувачів першого (бакалаврського) рівня вищої освіти під час виконання курсового проєкту, а також для формування системного бачення процесів забезпечення якості програмного забезпечення (ПЗ) на різних етапах його життєвого циклу.

У посібнику послідовно викладено підходи до планування та виконання основних видів тестування — модульного, інтеграційного, системного та валідаційного (приймального), а також наведено рекомендації щодо розроблення тестової документації, аналізу результатів тестування та формування обґрунтованих висновків. Особливу увагу приділено зв'язку тестування з вимогами до ПЗ та його відповідністю цілям і задачам кваліфікаційної роботи бакалавра.

Матеріали посібника орієнтовані не лише на засвоєння формальних процедур тестування, але й на розвиток інженерного мислення, здатності до критичного аналізу, відповідального ставлення до якості програмних продуктів і усвідомлення ролі тестування та верифікації у створенні надійних, безпечних і конкурентоспроможних програмних систем.

Автор(и) сподіваються, що даний посібник стане для здобувачів зручним практичним інструментом під час виконання курсового проєкту, сприятиме підвищенню якості підготовки бакалаврських робіт, а також закладе міцне підґрунтя для подальшої професійної діяльності у сфері розроблення, тестування та верифікації ПЗ.

Посібник може бути корисним також для викладачів, аспірантів і фахівців-практиків, які залучені до навчального процесу або працюють у галузі забезпечення якості програмного забезпечення, та може використовуватися як основа для подальшого удосконалення методичного забезпечення дисциплін, пов'язаних із розробленням і тестуванням програмних систем.

Курсовий проєкт з дисципліни **«Тестування та верифікація програмного забезпечення»** є важливою складовою професійної підготовки здобувачів першого (бакалаврського) рівня вищої освіти та спрямований на закріплення і поглиблення теоретичних знань, отриманих під час вивчення відповідного навчального курсу. Основною метою курсового проєкту є формування практичних навичок застосування сучасних методів, підходів і засобів тестування та верифікації програмного забезпечення, а також розвиток системного мислення, відповідальності та інженерної культури майбутнього фахівця з інформаційних технологій.

У межах виконання курсового проєкту здобувач навчається аналізувати вимоги до ПЗ, формалізувати їх у вигляді тестованих характеристик, обирати адекватні рівні та методи тестування, планувати процес

тестування, розробляти тестову документацію та інтерпретувати результати тестування з позицій якості, надійності та відповідності програмного продукту поставленим вимогам.

Курсовий проєкт виконується у **весняному семестрі четвертого курсу** та є логічно пов'язаним із кваліфікаційною роботою бакалавра. Тема курсового проєкту повинна повністю відповідати темі кваліфікаційної роботи бакалавра і формулюватися за зразком: **«Тестування програмного забезпечення [назва програмного забезпечення у родовому відмінку]»**. Такий підхід забезпечує цілісність підготовки здобувача, дозволяє використати результати курсового проєкту в кваліфікаційній роботі та сприяє комплексному опрацюванню розроблюваного програмного продукту.

Керівництво курсовим проєктом здійснюють викладачі, які проводять заняття з дисципліни **«Тестування та верифікація програмного забезпечення (КП)»**, що забезпечує методичну єдність викладання теоретичного матеріалу та його практичного застосування під час виконання проєкту.

У процесі виконання курсового проєкту здобувач повинен розробити повний комплект документації з тестування ПЗ, зокрема: переліки функціональних та нефункціональних вимог, верифікаційні таблиці, плани модульного, інтеграційного, системного та валідаційного (приймального) тестування. На основі розроблених планів необхідно виконати відповідні види тестування, зафіксувати результати у вигляді тестових звітів, проаналізувати виявлені дефекти та сформулювати обґрунтовані висновки щодо якості ПЗ.

Структура курсового проєкту повинна відповідати структурі **третього розділу «Тестування програмного забезпечення»** кваліфікаційної роботи бакалавра, що забезпечує узгодженість змісту та спрощує інтеграцію матеріалів курсового проєкту у фінальну роботу.

Загальний обсяг основної частини курсового проєкту рекомендується обмежити **30–35 сторінками** та включити такі структурні елементи:

- Вступ – до 1 сторінки;
- Планування валідаційного (приймального) тестування – до 3 сторінок;
- Планування системного тестування – до 5 сторінок;
- Планування інтеграційного тестування – до 4 сторінок;
- Планування модульного тестування – до 4 сторінок;
- Виконання модульного тестування – до 4 сторінок;
- Виконання інтеграційного тестування – до 4 сторінок;
- Виконання системного тестування – до 5 сторінок;
- Виконання валідаційного (приймального) тестування – до 3 сторінок;

- Висновки – до 1 сторінки.

Виконання курсового проєкту сприяє формуванню у здобувача практичних компетентностей з тестування та верифікації ПЗ, необхідних для подальшої професійної діяльності у сфері розроблення, аналізу та забезпечення якості програмних систем.

Сучасна модель якості ПЗ за стандартами ISO/IEC

Сучасна модель якості ПЗ визначена стандартом **ISO/IEC 25010:2011** *Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models*. Цей стандарт є частиною сімейства **ISO/IEC 25000 (SQuaRE)** і замінив застарілу модель **ISO/IEC 9126**, запропонувавши більш структурований та орієнтований на практику підхід до оцінювання якості програмних систем.
ISO/IEC 25010 визначає:

*модель **якості продукту** (Product Quality);*
*модель **якості у використанні** (Quality in Use).*

Модель якості продукту (Product Quality Model)

Модель якості продукту описує властивості ПЗ **незалежно від конкретного користувача**, з точки зору його архітектури, реалізації та поведінки. Вона включає **8 основних характеристик**, кожна з яких має підхарактеристики.

1. Functional Suitability (Функціональна придатність) здатність ПЗ надавати функції, які відповідають встановленим та задокументованим вимогам і є достатніми для досягнення поставлених цілей у визначеному контексті використання.

1.1 Functional completeness (повнота функціональності) характеризує ступінь того, як ПЗ реалізує усі функції, необхідні для виконання завдань, визначених у вимогах. Відсутність хоча б однієї критичної функції означає порушення повноти функціональності.

Приклад: *відсутність функції збереження даних у системі обліку*.

1.2 Functional correctness (функціональна правильність) описує здатність ПЗ коректно виконувати визначені функції, забезпечуючи правильні результати з допустимою точністю.

Приклад: *правильне обчислення результатів, відсутність логічних помилок, відповідність алгоритмів специфікації.*

1.3 Functional appropriateness (функціональна доцільність) визначає, наскільки реалізовані функції сприяють ефективному досягненню цілей користувача, не створюючи зайвих або надлишкових дій.

Приклад: *автоматизація рутинних операцій замість ручного введення.*

2. Продуктивність (Performance Efficiency) здатність ПЗ забезпечувати необхідний рівень продуктивності відносно використаних обчислювальних ресурсів за визначених умов.

2.1 Time behaviour (Часові характеристики) описують швидкодію системи, включаючи час відгуку, затримки, час виконання операцій та дотримання часових обмежень.

Особливо важливо для: систем реального часу, CPS, embedded-систем.

2.2 Resource utilization (використання ресурсів) характеризує ефективність використання процесора, пам'яті, дискового простору, мережевих ресурсів тощо.

Приклад: *мінімальне споживання оперативної пам'яті за зростання навантаження.*

2.3 Capacity (ємність, пропускна здатність, масштабованість) визначає максимальні межі навантаження, які система може обробляти без деградації якості.

Приклад: *кількість одночасних користувачів або транзакцій.*

3. Сумісність (Compatibility) Визначення здатність ПЗ ефективно співіснувати та взаємодіяти з іншими програмними або апаратними системами.

3.1 Co-existence (Співіснування) - здатність ПЗ працювати спільно з іншими системами в одному середовищі, не створюючи конфліктів у використанні ресурсів.

3.2 Interoperability (інтероперабельність) - здатність ПЗ обмінюватися даними та коректно їх інтерпретувати через стандартизовані інтерфейси та протоколи.

4. Зручність використання (Usability) - зручність використання визначає, наскільки ПЗ є зрозумілим, легким у навчанні та комфортним для користувача.

4.1 Appropriateness recognizability (зозпізнаваність придатності) - здатність користувача швидко зрозуміти, чи підходить ПЗ для виконання його завдань.

4.2 Learnability (Навчуваність) характеризує легкість, з якою користувач може навчитися працювати з системою.

4.3 Operability (керованість) визначає простоту та зручність виконання операцій, навігації та управління системою.

4.4 User error protection (захист від помилок користувача) - здатність ПЗ запобігати помилкам користувача або мінімізувати їх наслідки.

4.5 User interface aesthetics (естетика інтерфейсу користувача) характеризує візуальну привабливість, узгодженість та читабельність інтерфейсу.

4.6 Accessibility (доступність) - здатність системи бути доступною для користувачів з обмеженими можливостями.

5. Надійність (Reliability) - здатність ПЗ безперервно та коректно функціонувати у заданих умовах протягом визначеного часу.

5.1 Maturity (Зрілість) - ймовірність відсутності відмов у процесі експлуатації.

5.2 Availability (Доступність) - здатність ПЗ бути доступною для використання тоді, коли це необхідно.

5.3 Fault tolerance (стійкість до відмов) - здатність ПЗ продовжувати роботу при виникненні помилок або відмов.

5.4 Recoverability (відновлюваність) - здатність ПЗ відновлювати працездатність і дані після збою.

6. Безпека (Security) - здатність ПЗ захищати інформацію та ресурси від несанкціонованого доступу та зловмисних дій.

6.1 Confidentiality (конфіденційність) - забезпечення доступу до інформації лише для авторизованих користувачів.

6.2 Integrity (цілісність) - захист даних від несанкціонованих змін.

6.3 Non-repudiation (незаперечність) - неможливість заперечення факту виконання дії.

6.4 Accountability (підзвітність) - можливість однозначного встановлення відповідального суб’єкта.

6.5 Authenticity (автентичність) - підтвердження справжності користувачів або компонентів.

7. Супроводжуваність (Maintainability) - здатність ПЗ бути проаналізованим, зміненим і перевіреним з мінімальними зусиллями.

7.1 Modularity (модульність) - ступінь розділення ПЗ на незалежні компоненти.

7.2 Reusability (повторне використання) - здатність компонентів бути використаними повторно.

7.3 Analysability (аналізованість) - легкість виявлення причин дефектів.

7.4 Modifiability (модифікованість) - легкість внесення змін.

7.5 Testability (тестованість) - здатність ПЗ бути ефективно протестованою.

8. Переносимість (Portability) - здатність ПЗ бути встановленим і функціонувати в різних апаратно-програмних середовищах.

8.1 Adaptability (адаптованість) - системи пристосовуватися до різних середовищ без змін.

8.2 Installability (інстальованість) - легкість встановлення та видалення.

8.3 Replaceability (замінність) - здатність бути заміненим іншим продуктом.

Модель якості у використанні (Quality in Use Model)

Ця модель описує якість ПЗ **з точки зору користувача та контексту використання**.

Модель якості у використанні (Quality in Use) описує ступінь, у якому програмне забезпечення дозволяє користувачам ефективно, безпечно та з задоволенням досягати поставлених цілей у конкретному контексті використання. На відміну від якості продукту, ця модель орієнтована не на внутрішні властивості ПЗ, а на результат його реального застосування.

1. *Effectiveness* (ефективність досягнення цілей користувача) характеризує ступінь, у якому користувачі успішно та повно досягають своїх цілей за допомогою ПЗ з необхідною точністю та повнотою.

Ключові аспекти:

повнота виконання завдань;
точність отриманих результатів;
відсутність критичних помилок у процесі використання;
відповідність результатів очікуванням користувача.

Приклади показників:

відсоток успішно завершених сценаріїв;
кількість помилок, що заважають досягненню мети;
точність обчислень або рішень.

Зв’язок із тестуванням:

Оцінюється переважно під час валідаційного (приймального) тестування, сценарного тестування та тестування з участю користувачів.

2. Efficiency (ефективність використання ресурсів під час експлуатації) описує, наскільки ефективно завдяки використанню ПЗ користувачі можуть досягати своїх цілей за мінімальних витрат часу, зусиль і ресурсів

Ключові аспекти:

час виконання типових операцій;
кількість дій, необхідних для досягнення результату;
когнітивне навантаження на користувача;
споживання системних ресурсів у процесі використання.

Приклади показників:

середній час виконання завдання;
кількість кліків або кроків;
затримки реакції системи.

Зв’язок із тестуванням:

Оцінюється під час юзабіліті-тестування, валідаційного тестування та аналізу поведінки користувачів.

3. Satisfaction (задоволеність користувача) відображає суб'єктивне сприйняття користувачем ПЗ, зокрема рівень комфорту, довіри та загального позитивного досвіду під час його використання.

Ключові аспекти:

задоволення від взаємодії з інтерфейсом;
відповідність очікуванням;
відчуття контролю та впевненості;
готовність рекомендувати продукт.

Приклади показників:

опитування користувачів;
індекси задоволеності (CSAT, SUS);
частота добровільного використання.

Зв'язок із тестуванням:

Оцінюється під час приймального тестування, пілотної експлуатації та тестування з реальними користувачами.

4. Freedom from risk (свобода від ризиків) характеризує ступінь, у якому використання ПЗ не створює неприпустимих ризиків для користувачів, організацій, навколишнього середовища або інших зацікавлених сторін.

Види ризиків:

ризики безпеки (втрата або компрометація даних);
фінансові ризики (помилкові операції, штрафи);
ризики для здоров'я та безпеки людей;
екологічні ризики (особливо у CPS).

Приклади показників:

кількість інцидентів;
рівень критичності наслідків помилок;
відповідність нормативним вимогам.

Зв'язок із тестуванням:

Перевіряється за допомогою валідаційного тестування, тестування безпеки, тестування відмов та аналізу ризиків.

5. Context coverage (покриття контекстів використання) відображає здатність ПЗ ефективно функціонувати у різних реальних умовах використання, включаючи різні типи користувачів, середовища та сценарії.

Ключові аспекти:

різні профілі користувачів;
різні пристрої та платформи;
різні умови експлуатації;
нестандартні або граничні сценарії.

Приклади показників:

кількість підтримуваних сценаріїв;

стабільність роботи у змінних умовах;

адаптивність інтерфейсу.

Зв'язок із тестуванням:

Оцінюється під час сценарного тестування, кросплатформного тестування та валідації у різних умовах експлуатації.

Характеристики Quality in Use дозволяють оцінити не лише формальну відповідність ПЗ вимогам, а й реальну його цінність для користувача. Саме ці характеристики є визначальними на етапі приймання ПЗ та підтвердження готовності до практичного використання.

Модель ISO/IEC 25010 широко використовується як:

- **основа для формування вимог до якості**;
- джерело **тестованих атрибутів якості**;
- каркас для **валідаційних і верифікаційних критеріїв**;
- база для побудови **трасування «вимога → характеристика якості → тест»**.

У практиці тестування:

- **верифікація** перевіряє відповідність характеристик продукту вимогам (Product Quality);
- **валідація** оцінює відповідність якості очікуванням користувача (Quality in Use).

Модель ISO/IEC 25010 є сучасною:

- враховує **безпеку та доступність** як повноцінні характеристики;
- підтримує **системи та CPS**, а не лише ПЗ;
- добре узгоджується з **Agile, DevOps, CI/CD**;
- застосовується в **safety-critical та mission-critical системах**;
- легко інтегрується у **Model-Based Design та вимогову інженерію**.

ISO/IEC 25010 задає універсальну, ієрархічну та практично орієнтовану модель якості ПЗ, яка дозволяє:

- формалізувати поняття «якість ПЗ»;
- пов'язати вимоги, тестування та оцінювання;
- забезпечити для ПЗ об'єктивну верифікацію та валідацію.

Зв'язок характеристик якості ISO/IEC 25010 з типами тестів

1. *Функціональна придатність (Functional Suitability)*

Мета: перевірити, що ПЗ реалізує всі вимоги коректно та доцільно.

Типи тестів:

Функціональне тестування

Системне тестування

Приймальне (валідаційне) тестування
Тестування вимог (requirements-based testing)
Перевіряються підхарактеристики:
Functional completeness → тестування повноти вимог
Functional correctness → позитивні та негативні тест-кейси
Functional appropriateness → сценарне та користувацьке тестування

2. *Продуктивність (Performance Efficiency)*
Мета: оцінити часові характеристики та використання ресурсів.
Типи тестів:
Навантажувальне тестування (Load testing)
Стрес-тестування (Stress testing)
Тестування продуктивності (Performance testing)
Тестування масштабованості
Перевіряються підхарактеристики:
Time behaviour - вимірювання часу відгуку
Resource utilization - профілювання CPU, RAM, I/O
Capacity - граничні та пікові навантаження
3. *Сумісність (Compatibility)*
Мета: перевірити коректну взаємодію з іншими системами.
Типи тестів:
Інтеграційне тестування
Тестування інтерфейсів (API testing)
Тестування сумісності (Compatibility testing)
Перевіряються підхарактеристики:
Co-existence - паралельна робота з іншими системами
Interoperability - обмін даними та коректність протоколів
4. *Зручність використання (Usability)*
Мета: оцінити зручність, зрозумілість та доступність системи.
Типи тестів:
Тестування зручності використання (Usability testing)
Юзабіліті-інспекції
Експертна оцінка UI
Тестування доступності (Accessibility testing)
Перевіряються підхарактеристики:
Learnability → тестування першого використання
Operability → сценарії щоденної роботи
User error protection → негативні сценарії
Accessibility → відповідність WCAG
5. *Надійність (Reliability)*
Мета: забезпечити стабільну роботу системи у часі та при збоях.
Типи тестів:
Тестування надійності (Reliability testing)

Тестування безперервної роботи (Soak testing)
Тестування відмовостійкості
Тестування відновлення (Recovery testing)

Перевіряються підхарактеристики:

Maturity → статистика відмов
Availability → тестування доступності
Fault tolerance → ін'єкція помилок
Recoverability → відновлення після збоїв

6. *Безпека (Security)*

Мета: захистити дані та ресурси системи.

Типи тестів:

Тестування безпеки (Security testing)
Пентестинг
Тестування аутентифікації та авторизації
Аналіз вразливостей

Перевіряються підхарактеристики:

Confidentiality → контроль доступу
Integrity → захист від змін
Non-repudiation → аудит і логування
Authenticity → перевірка автентифікації

7. *Супроводжуваність (Maintainability)*

Мета: забезпечити можливість ефективного супроводу та розвитку ПЗ.

Типи тестів:

Модульне тестування
Статичний аналіз коду
Рецензування коду
Регресійне тестування

Перевіряються підхарактеристики:

Modularity → ізольованість модулів
Analysability → логування, діагностика
Modifiability → простота внесення змін
Testability → можливість автоматизації

8. *Переносимість (Portability)*

Мета: перевірити роботу ПЗ в різних середовищах.

Типи тестів:

Тестування переносимості
Кросплатформне тестування
Інсталяційне тестування
Конфігураційне тестування

Перевіряються підхарактеристики:

Adaptability → робота в різних ОС
Installability → встановлення/оновлення

Replaceability → заміна компонентів

Таблиця 1 Характеристики якості ПЗ та типи тестів

Характеристика ISO/IEC 25010	**Основні типи тестів**
Functional suitability	Функціональне, системне, приймальне
Performance efficiency	Навантажувальне, стрес, продуктивності
Compatibility	Інтеграційне, API
Usability	*Usability* , accessibility
Reliability	Надійності, відновлення
Security	Безпеки, пентестинг
Maintainability	Модульне, статичний аналіз
Portability	Кросплатформне, інсталяційне

Таким чином, модель якості ISO/IEC 25010 безпосередньо визначає набір та обсяг тестових робіт, необхідних для верифікації та валідації програмного забезпечення.

Функціональне, системне та приймальне тестування

Функціональне тестування спрямоване на перевірку відповідності реалізованих функцій ПЗ функціональним вимогам, визначеним у специфікації вимог (SRS). Перевіряється коректність виконання алгоритмів, оброблення вхідних даних, формування вихідних результатів та реакції системи на допустимі і недопустимі дії. Функціональне тестування особливо важливе для перевірки:

алгоритмів керування;
оброблення фізичних величин;
коректності логіки прийняття рішень.

Системне тестування виконується для перевірки ПЗ як цілісної системи, інтегрованої з усіма підсистемами та середовищем виконання. На цьому етапі оцінюється відповідність ПЗ системним вимогам, а також нефункціональним характеристикам.

Для safety-critical систем системне тестування включає:

перевірку роботи в штатних і аварійних режимах;
тестування часових обмежень;
оцінювання стабільності та детермінованості поведінки.

Приймальне (валідаційне) тестування проводиться з метою підтвердження того, що ПЗ відповідає очікуванням замовника та готове до експлуатації. Воно базується на сценаріях реального використання та критеріях приймання.

Приймальне тестування дозволяє:

підтвердити досягнення цілей системи;

обґрунтувати допуск ПЗ до експлуатації;

прийняти рішення щодо завершення розроблення.

Навантажувальне, стрес-тестування та тестування продуктивності

Навантажувальне тестування перевіряє здатність ПЗ коректно працювати при номінальному та піковому навантаженні. Аналізуються показники часу відгуку, пропускної здатності та використання ресурсів. Це важливо для:

систем реального часу;

оброблення потоків сенсорних даних;

багатоканальних систем керування.

Стрес-тестування є розширенням навантажувального тестування та спрямоване на перевірку поведінки системи за межами допустимих умов експлуатації. Метою є визначення точок відмови та аналізування стабільності системи.

Тестування продуктивності оцінює здатність ПЗ забезпечувати задані часові характеристики та ефективно використовувати ресурси. Для авіаційних систем це критично з точки зору дотримання вимог реального часу.

Інтеграційне тестування та тестування API

Інтеграційне тестування спрямоване на перевірку коректності взаємодії між модулями та підсистемами, дозволяє виявити помилки, що виникають на стиках програмних і апаратних компонентів.

Тестування API полягає у перевірці коректності інтерфейсів взаємодії між компонентами системи або із зовнішніми системами. Перевіряється формат даних, протоколи обміну, обробка помилок та часові характеристики.

Юзабіліті- та accessibility-тестування

Юзабіліті-тестування (usability testing) оцінює зручність, інтуїтивність та ефективність взаємодії користувача з системою, спрямоване на зменшення ймовірності помилок оператора, спрямоване на оцінювання того, наскільки ПЗ є зрозумілим, зручним та ефективним для користувача під час виконання типових задач. Тобто «Чи може користувач легко, швидко і без помилок досягти своєї мети, працюючи з ПЗ?»:

користувачі або тестувальники виконують реальні сценарії;
спостерігається їхня поведінка;
фіксуються труднощі, помилки, непорозуміння;
оцінюється суб'єктивна задоволеність користувачів.

Акцент робиться не на коректності алгоритмів, а на взаємодії людини з системою.

Юзабіліті-тестування напряму пов'язане з характеристикою Usability (Зручність використання), яка включає:

Appropriateness recognizability — зрозумілість призначення системи;
Learnability — легкість навчання;
Operability — зручність керування;
User error protection — захист від помилок користувача;
User interface aesthetics — естетика інтерфейсу;
Accessibility — доступність для різних категорій користувачів.

Приклад юзабіліті-тесту
Завдання для користувача:
«Створіть новий обліковий запис та змініть пароль».
Під час тесту фіксується:

чи зрозуміло, з чого почати;
кількість кліків і часу виконання;
кількість помилок;
моменти, де користувач вагається;
суб'єктивна оцінка зручності.

Типи юзабіліті-тестування

Модероване — з участю експерта
Немодероване — самостійна робота користувача
Лабораторне — у контрольованих умовах
Польове — у реальному середовищі
Експертне (heuristic evaluation) — аналіз за евристиками
A/B-тестування — порівняння варіантів інтерфейсу

Показники (метрики) юзабіліті

час виконання завдання;
кількість помилок;
кількість звернень до довідки;
рівень завершення завдань (%);
задоволеність користувачів (SUS, CSAT).

Таблиця 2 Переваги та обмеження юзабіліті-тестування

Переваги	Обмеження
підвищує задоволеність користувачів зменшує кількість помилок знижує витрати на навчання підвищує конкурентоспроможність ПЗ виявляє проблеми ще до релізу	потребує участі реальних користувачів результати можуть бути суб'єктивними не замінює функціональне тестування

Юзабіліті-тестування — вид тестування ПЗ, спрямований на оцінювання зручності, зрозумілості, ефективності та задоволеності користувачів під час взаємодії з системою з метою забезпечення відповідності ПЗ вимогам ергономіки та стандартам якості.

Accessibility-тестування (тестування доступності) перевіряє доступність ПЗ для користувачів з обмеженими можливостями та відповідність стандартам доступності, що підвищує універсальність та безпечність використання системи.

Accessibility-тестування — вид нефункціонального тестування ПЗ, спрямований на перевірку того, що ПЗ є доступним для максимально широкого кола користувачів, зокрема осіб з тимчасовими або постійними обмеженнями зору, слуху, моторики чи когнітивних можливостей.

Основна мета: перевірка можливості безпечного, ефективного та рівноправного використання ПЗ всіма користувачами незалежно від їхніх фізичних можливостей та умов експлуатації.

Місце Accessibility у моделі якості ISO/IEC 25010

Accessibility є підхарактеристикою Usability (Зручність використання) та безпосередньо пов'язана з:

Effectiveness — здатність користувача досягти цілей;

Freedom from risk — зменшення ризику помилок оператора;

Context coverage — робота в різних умовах використання.

У safety-critical системах доступність інтерфейсу має прямий вплив на безпеку експлуатації.

Для систем критичного призначення accessibility-тестування є критично важливим, оскільки:

оператор може працювати в умовах стресу, втоми або обмеженої уваги;

інтерфейс використовується в реальному часі;

помилки взаємодії можуть призвести до небезпечних наслідків;

система має бути зрозумілою незалежно від досвіду оператора.

Основні аспекти Accessibility-тестування:

1. Візуальна доступність

Перевіряється, чи може користувач із порушеннями зору коректно сприймати інформацію:

достатній контраст кольорів;
масштабування тексту без втрати змісту;
читабельність шрифтів;
відсутність критичної інформації, поданої лише кольором;
коректна робота у режимах високого контрасту.

2. Клавіатурна та навігаційна доступність

Перевіряється можливість повноцінної роботи без використання миші:

навігація за допомогою клавіатури;
логічний порядок фокусу;
наявність видимого індикатора фокусу;
відсутність «пасток фокусу».

Це особливо важливо для вбудованих та промислових систем керування.

3. Доступність для допоміжних технологій

Оцінюється сумісність ПЗ з:

screen readers;
системами голосового керування;
альтернативними пристроями введення.
Перевіряється:
наявність семантичних описів;
коректна ідентифікація елементів інтерфейсу;
доступність повідомлень про помилки.

4. Когнітивна доступність

Спрямована на зменшення навантаження на користувача:
зрозумілі формулювання повідомлень;
однозначність інструкцій;
відсутність перевантаження інформацією;
передбачувана поведінка інтерфейсу.

У safety-critical системах це знижує ймовірність людської помилки.

5. Доступність у нестандартних умовах

Перевіряється робота інтерфейсу:
при яскравому освітленні;
у шумному середовищі;
при обмеженому часі реакції;
на різних екранах і пристроях.

Таблиця 3 Аспекти доступності та методи їх перевірки

Аспект доступності	**Що перевіряється**	**Метод перевірки**	**Очікуваний результат**
Візуальна доступність	Контраст кольорів, читабельність тексту, масштабування	Ручне тестування, експертна	Інформація чітко сприймається за різних умов освітлення

	інтерфейсу	оцінка за WCAG	
Колірна незалежність	Передавання інформації не лише кольором	Ручна перевірка, симуляція дальтонізму	Критичні стани зрозумілі без використання кольору
Масштабування та адаптивність	Коректність відображення при збільшенні інтерфейсу	Ручне тестування	Інтерфейс не втрачає функціональність
Клавіатурна доступність	Повна керованість без миші	Ручне тестування з клавіатурою	Усі функції доступні з клавіатури
Навігація та фокус	Логічний порядок переходу між елементами	Ручне тестування, чек-листи	Фокус переміщується передбачувано
Підтримка допоміжних технологій	Сумісність зі screen readers	Ручне тестування з допоміжними засобами	Інтерфейс коректно озвучується
Семантика інтерфейсу	Коректне позначення елементів керування	Інспекція коду, експертна оцінка	Елементи правильно ідентифікуються
Повідомлення про помилки	Зрозумілість та інформативність повідомлень	Сценарне тестування	Користувач чітко розуміє причину помилки
Когнітивна доступність	Простота сприйняття інформації	Юзабіліті-тестування	Мінімізується когнітивне навантаження
Часові обмеження	Можливість керування часом реакції	Сценарне тестування	Користувач встигає виконати дію
Аудіо-доступність	Наявність візуальних альтернатив звукових сигналів	Ручна перевірка	Важлива інформація не залежить від звуку
Робота в складних умовах	Читабельність у стресових умовах	Імітаційне тестування	Інтерфейс залишається зрозумілим
Консистентність інтерфейсу	Однаковість елементів керування	Експертна оцінка	Поведінка інтерфейсу передбачувана
Документація та підказки	Доступність довідкової інформації	Ручне тестування	Користувач може швидко отримати допомогу

Для критичних **систем** особлива увага приділяється аспектам, пов’язаним із **зниженням ймовірності помилок оператора**, зокрема когнітивній доступності, зрозумілості повідомлень про помилки та роботі інтерфейсу в умовах обмеженого часу прийняття рішень.

Стандарти та нормативна база

Accessibility-тестування зазвичай орієнтується на такі стандарти:
WCAG 2.1 / 2.2 (Web Content Accessibility Guidelines);
ISO/IEC 25010 (Usability → Accessibility);
EN 301 549 (для критичних систем та публічного сектору);
рекомендації з Human Factors Engineering (особливо в авіації).
Методи Accessibility-тестування

Ручне тестування — експертна перевірка інтерфейсів
Тестування з користувачами — реальні сценарії використання
Автоматизоване тестування — перевірка типових порушень
Експертна оцінка — аналіз відповідності стандартам

Для критичних систем автоматизовані методи не замінюють ручне тестування.
Типові дефекти, що виявляються

недостатній контраст;
відсутність альтернативних описів;
нелогічна навігація;
складні або неінформативні повідомлення про помилки;
залежність від одного способу введення.

Accessibility-тестування — це вид нефункціонального тестування ПЗ, спрямований на перевірку доступності, зручності та безпечності взаємодії з системою для користувачів з різними фізичними та когнітивними можливостями відповідно до стандартів якості та ергономіки.

Accessibility-тестування - невід'ємна складова забезпечення якості сучасного ПЗ, особливо у кіберфізичних та safety-critical системах, де помилки взаємодії людини з системою можуть мати критичні наслідки. Забезпечення доступності підвищує надійність, безпеку та ефективність експлуатації ПЗ, а також відповідає сучасним вимогам міжнародних стандартів.

Тестування надійності та відновлення

Тестування надійності (Reliability Testing) перевіряє здатність системи безперервно виконувати свої функції протягом заданого часу без відмов. Для CPS це пов'язано з вимогами безпеки та відмовостійкості. Тестування надійності — це вид нефункціонального тестування програмного забезпечення, спрямований на перевірку здатності системи коректно працювати протягом визначеного часу та умов експлуатації без відмов. У критичних системах надійність є однією з ключових характеристик якості, оскільки будь-яка відмова може мати серйозні наслідки для життя людей, обладнання та навколишнього середовища.

Мета тестування надійності: Перевірити, чи ПЗ може безперервно виконувати свої функції протягом заданого проміжку часу. Виявити та

локалізувати дефекти, які проявляються під час тривалої роботи. Оцінити середній час до відмови (MTTF) та здатність системи до відновлення після збоїв. Забезпечити впевненість у відповідності ПЗ вимогам до надійності та безпечної експлуатації.

Таблиця 4 Основні параметри надійності

Параметр	Опис
MTTF (Mean Time To Failure)	Середній час безвідмовної роботи системи
MTTR (Mean Time To Repair)	Середній час відновлення працездатності після збоїв
Availability	Доля часу, протягом якого система доступна та працездатна
Fault tolerance	Здатність системи продовжувати роботу при часткових відмовах

Види тестування надійності

Тривале (Endurance / Soak Testing) - перевірка стабільності ПЗ при тривалій роботі в штатних умовах. Наприклад: симуляція безперервної роботи авіоніки протягом кількох годин або днів.

Випробування на відмову (Failure Testing) –цілеспрямоване створення відмов компонентів (сенсорів, актуаторів, мережевих каналів) для перевірки поведінки системи.

Випробування відновлення (Recovery Testing) - перевірка здатності системи відновлювати працездатність після аварійних станів, включаючи перезапуск компонентів, відновлення даних або переключення на резервні модулі.

Сценарне надійнісне тестування - використання реалістичних сценаріїв експлуатації з комбінованими навантаженнями та відмовами для оцінки стабільності роботи CPS у складних умовах.

Методи проведення тестування надійності

Симуляційне тестування (Software-in-the-Loop / Hardware-in-the-Loop) — імітація роботи системи в умовах реальної експлуатації.

Моніторинг у реальному часі — збір даних про відмови та поведінку системи під навантаженням.

Статистичний аналіз — обчислення MTTF, MTTR, показників доступності та ймовірності відмов.

Fault injection — цілеспрямоване введення помилок для оцінки стабільності та відмовостійкості.

Значення для критично важливих систем

Авіоніка та космічні системи: перевірка стабільності алгоритмів керування, сенсорних та обчислювальних підсистем під час польоту.

Медичні пристрої: гарантування безперервного функціонування апаратів життєзабезпечення.

Автоматизовані промислові системи: оцінка роботи керуючих програм при тривалій експлуатації та непередбачуваних ситуаціях.

Результати тестування надійності: Підтвердження того, що система відповідає вимогам надійності. Ідентифікація слабких місць, які потребують доопрацювання. Формування статистичних показників для сертифікації ПЗ у критичних системах.

Тестування відновлення (Recovery Testing) оцінює здатність системи відновлювати працездатність після збоїв, зокрема після втрати живлення, відмови компонентів або програмних помилок.

Тестування відновлення — вид нефункціонального тестування, спрямований на перевірку здатності ПЗ відновлювати працездатність після відмов, збоїв або аварійних ситуацій.

Мета тестування – зменшення ризику тривалих простоїв та забезпечення безпеки роботи системи. Перевірити, чи ПЗ може швидко та коректно відновлювати функції після збоїв. Оцінити ефективність механізмів відновлення, таких як резервування, відкат до попереднього стану або переключення на дублюючі модулі. Забезпечити відповідність вимогам безпеки та надійності для критично важливих систем.

Таблиця 5 Основні аспекти тестування відновлення

Аспект	Що перевіряється
Відновлення після апаратних збоїв	Перезапуск процесів, робота з резервними сенсорами або актуаторами
Відновлення після програмних збоїв	Перезапуск програмного модуля, відкат до стабільного стану
Збереження даних	Захист від втрати даних під час аварій, правильне відновлення бази даних або журналу подій
Час відновлення (Recovery Time)	Визначення максимально допустимого часу, необхідного для повернення системи в працездатний стан
Дотримання безпечного стану	Підтвердження, що під час відновлення система не створює небезпечних умов для користувачів або обладнання

Методи тестування відновлення

Fault injection (введення помилок): навмисне створення збоїв у сенсорах, актуаторах або програмних модулях для перевірки реакції системи.

Симуляція відмов компонентів: імітація відмов апаратних або програмних підсистем у безпечному середовищі (Model-in-the-Loop (MiL), Software-in-the-Loop (SiL) та Hardware-in-the-Loop (HiL)).
Перевірка резервування та дублювання: тестування роботи ПЗ при переключенні на резервні канали, модулі або копії даних.
Стрес та тривале тестування збоїв: Оцінка поведінки ПЗ при повторюваних або тривалих відмовах, щоб перевірити стабільність механізмів відновлення.

Значення для критично – важливих систем
Авіоніка та космічні системи: гарантує, що після збою бортові контролери або системи управління повертаються в безпечний режим.
Медичні пристрої: забезпечує відновлення життєзабезпечувальних функцій після збоїв.
Промислова автоматизація: дозволяє ПЗпродовжувати роботу або безпечно зупинитися під час аварій.

Результати тестування відновлення:

Визначення швидкості та ефективності відновлення ПЗ після різних типів збоїв.

Виявленння слабких місць в механізмах резервування та дублювання.

Підтвердженння того, що ПЗ може безпечно відновитися та продовжувати роботу або перейти в безпечний стан.

Тестування відновлення є критично важливим для safety-critical систем, оскільки гарантує безпеку, безперервність роботи та мінімізацію наслідків збоїв. У CPS та авіації це дозволяє забезпечити надійне функціонування навіть у надзвичайних ситуаціях, що є ключовим критерієм сертифікації та експлуатації.

Тестування безпеки та пентестинг

Тестування безпеки спрямоване на перевірку захищеності даних, каналів зв’язку та механізмів доступу. У CPS особлива увага приділяється захисту від несанкціонованого керування фізичними процесами.

Пентестинг (penetration testing) полягає в імітації атак зловмисника з метою виявлення вразливостей. Проводиться контрольовано та відповідно до нормативних обмежень.

Модульне тестування та статичний аналіз

Модульне тестування забезпечує перевірку окремих функцій та класів у ізольованому середовищі. Воно є основою побудови надійної тестової піраміди.

Статичний аналіз виконується без запуску програми та спрямований на виявлення потенційних дефектів у коді, порушень стандартів програмування та небезпечних конструкцій. У критично-важливих системах статичний аналіз є обов'язковим елементом верифікації.

Кросплатформене та інсталяційне тестування

Кросплатформене тестування перевіряє здатність ПЗ коректно працювати на різних апаратних та програмних платформах. Це особливо важливо для розподілених систем. Під платформою у кросплатформеному тестуванні розуміють комбінацію:

операційної системи (Windows, Linux, macOS, Android, iOS);
апаратної архітектури (x86, x64, ARM);
середовища виконання (браузер, JVM, .NET, контейнер);
версії ОС або середовища;
типу пристрою (ПК, ноутбук, смартфон, планшет).

Інсталяційне тестування оцінює коректність процесу встановлення, оновлення та видалення ПЗ. Для критичних систем гарантує, що ПЗ може бути коректно розгорнуте без порушення вимог безпеки.

Застосування зазначених типів тестування дозволяє забезпечити всебічну перевірку ПЗ, охоплюючи всі ключові характеристики якості відповідно до стандарту ISO/IEC 25010. Такий підхід є необхідним для створення надійних, безпечних та сертифікованих CPS і авіаційних систем, готових до практичного використання.

Системи критичного призначення

Safety-critical системи (системи з критичною важливістю для безпеки) — це системи, від надійної та безпечної роботи яких безпосередньо залежать життя людей, здоров'я, навколишнє середовище або запобігання значних матеріальних збитків. Вихід з ладу або неправильність роботи системи може призвести до серйозних наслідків, включно з травмами, смертю, аваріями або екологічними катастрофами.

Основні характеристики safety-critical систем

1 Високі вимоги до надійності

ПЗ має працювати безвідмовно протягом визначеного часу;
відмови повинні бути передбачуваними та контрольованими.

2 Сертифікація та відповідність стандартам

у авіації: DO-178C, DO-254;
у автомобільній індустрії: ISO 26262;
у промислових системах: IEC 61508.

3 Верифікація та валідація на всіх рівнях
обов'язкове модульне, інтеграційне, системне та приймальне тестування;
аналіз сценаріїв аварійних ситуацій;
документування всіх етапів тестування.

4 Відмовостійкість та безпечне завершення роботи
система повинна або продовжувати роботу в безпечному режимі, або безпечно зупиняти процеси при збоях.

5 Надання інформації оператору
інтерфейси повинні бути зрозумілими та зручними;
забезпечується зменшення ймовірності людської помилки.

Таблиця 6Приклади safety-critical систем

Галузь	Приклади
Авіація	Авіоніка літаків, системи управління польотом, диспетчеризація регламентних робіт
Космічні системи	Системи орієнтації та стабілізації супутників, керування апаратурою космічного корабля
Автомобільна промисловість	Системи керування гальмами ABS, електронні системи управління рухом (ESC), автономне керування
Промислова автоматизація	Аварійне зупинення верстатів, контроль хімічних процесів
Медичні пристрої	Дихальні апарати, інфузійні насоси, апарати життєзабезпечення

У критично-важливих системах кожен дефект може бути критичним, тому тестування має бути формалізованим і багаторівневим.
Всі зміни в коді потребують повторної верифікації.
Часто застосовуються моделювання, симуляції та Hardware-in-the-Loop (HiL) для перевірки роботи у складних умовах.

Вибір типів тестування для критичних систем

ПЗ кіберфізичних, авіаційних та інших систем критичного призначення (транспортних, енергетичних, військових тощо) функціонує в умовах жорстких обмежень щодо надійності, детермінованості, безпеки та відмовостійкості, а помилки в його роботі можуть призвести до значних матеріальних збитків, порушення функціонування технічних об'єктів або

загрози життю людей. У зв'язку з цим тестування такого ПЗ це багаторівневий, системний та формалізований проц3ес, що відповідає вимогам міжнародних стандартів (ISO/IEC 25010, DO-178C, ISO 26262, IEC 61508) та охоплювати як функціональні, так і нефункціональні аспекти якості.

З урахуванням специфіки систем критичного призначення було обґрунтовано застосування таких типів тестування.

Модульне тестування є базовим етапом перевірки ПЗ safety-critical систем і застосовується для верифікації окремих алгоритмічних та керуючих компонентів, зокрема:

алгоритмів оброблення сенсорних даних;
фільтрації, нормалізації та масштабування фізичних величин;
керуючих та регуляторних алгоритмів.

Даний тип тестування дозволяє:

виявляти помилки на ранніх етапах життєвого циклу;
забезпечити детермінованість виконання;
підвищити надійність та супроводжуваність коду, що є критично важливим для сертифікації авіаційного ПЗ.

Інтеграційне тестування спрямоване на перевірку коректності взаємодії ПЗ між собою та з фізичними підсистемами, такими як сенсори, актуатори, виконавчі механізми та канали зв'язку.

Особлива увага приділяється:

коректності обміну даними в реальному часі;
узгодженості одиниць вимірювання та форматів даних;
синхронізації між підсистемами.

Інтеграційне тестування є критично важливим для виявлення системних дефектів, які не можуть бути виявлені під час модульного тестування.

Системне тестування застосовується для оцінювання ПЗ як цілісної системи, що функціонує у визначеному експлуатаційному середовищі. Перевіряється відповідність ПЗ системним вимогам (System Requirements Specification), зокрема щодо:

функціональної повноти та коректності;
часових характеристик і реакції на події;
поведінки системи в штатних і нештатних режимах;
взаємодії з зовнішніми системами.

Системне тестування часто виконується із використанням MiL, SiL та HiL підходів.

Валідаційне (приймальне) тестування обрано для підтвердження того, що ПЗ реалізує саме ті функції та режими роботи, які необхідні для

безпечної експлуатації відповідно до вимог замовника та нормативної документації.

Валідаційне тестування спрямоване на:

перевірку досягнення цілей системи в реальних або наближених до реальних умовах;

підтвердження безпечної поведінки у критичних ситуаціях;

обґрунтування готовності ПЗ до введення в експлуатацію.

Сценарне тестування відіграє ключову роль у перевірці поведінки ПЗ у типових та аварійних сценаріях, зокрема:

зміні режимів роботи;

відмові сенсорів або актуаторів;

затримці або втраті даних;

перевищенні граничних параметрів.

Даний тип тестування дозволяє оцінити реакцію системи в динаміці та підтвердити коректність реалізації логіки керування в реальних умовах експлуатації.

Юзабіліті-тестування

Незважаючи на те, що авіаційні та safety-critical системи часто орієнтовані на професійних операторів, юзабіліті-тестування є необхідним для забезпечення:

зниження імовірності людських помилок;

зрозумілості інтерфейсів керування;

ефективної взаємодії оператора з системою в умовах стресу та обмеженого часу.

Юзабіліті-тестування безпосередньо пов’язане з характеристиками Usability та Freedom from risk стандарту ISO/IEC 25010.

Кросплатформене тестування застосовується для підтвердження переносимості ПЗ між різними обчислювальними платформами, операційними системами та апаратними архітектурами, що є актуальним для ПЗ з розподіленою архітектурою.

Це дозволяє:

зменшити ризики при заміні апаратних компонентів;

забезпечити довготривалу експлуатацію системи;

підвищити гнучкість модернізації.

Ці типи тестування забезпечують багаторівневу та комплексну перевірку ПЗ, охоплюючи всі критичні аспекти якості: функціональність, надійність, безпеку, детермінованість і зручність експлуатації. Такий підхід відповідає сучасним вимогам до розроблення та сертифікації safety-critical

ПЗ та дозволяє обґрунтовано стверджувати про готовність ПЗ до безпечного використання.

Вступ до курсового проєкту

Вступ до курсового проєкту має бути таким, щоб його можливо було використати як вступ до третього розділу бакалаврської кваліфікаційної роботи. Вступ має визначити важливість тестування для створення високоякісного ПЗ, стратегію тестування ПЗ, рівні тестування ПЗ та інструментальні засоби тестування. Мета цього підрозділу – пояснити роль тестування у життєвому циклі ПЗ, його місце в процесі розроблення та обґрунтувати вибір рівнів тестування (модульного, інтеграційного, системного, приймального).

Рекомендована структура і зміст підрозділу «Вступ»:

1. Загальна роль тестування у розробленні ПЗ

Слід пояснити, що тестування є невід'ємною складовою життєвого циклу розроблення ПЗ, а мета тестування - виявлення дефектів, перевірка відповідності вимогам і забезпечення якості ПЗ. Вказати, що тестування це не лише перевірка роботи ПЗ, а й процес підтвердження його надійності, безпеки, ефективності.

Приклад формулювання: Тестування ПЗ є важливою стадією життєвого циклу розроблення ПЗ, метою якої є перевірка правильності реалізації вимог, виявлення помилок і оцінювання якості ПЗ. На цьому етапі визначається ступінь готовності ПЗ до експлуатації та її відповідність функціональним і нефункціональним вимогам.

2. Місце тестування у життєвому циклі ПЗ

Слід описати, на якому етапі виконується тестування відносно проектування, кодування та впровадження. Зазначити, що тестування проводиться на всіх рівнях - від перевірки окремих модулів до перевірки повного ПЗ.

Приклад: Тестування виконується після етапу реалізації, але результати тестів враховуються протягом усього процесу розроблення. Для підвищення ефективності застосовується багаторівневий підхід: модульне, інтеграційне, системне та приймальне тестування.

3. Обґрунтування вибору рівнів тестування

Слід пояснити, що кожен рівень тестування має свою мету та об'єкт перевірки:

- модульне тестування призначене для перевірки окремих функцій і класів;
- інтеграційне тестування призначене для перевірки взаємодії компонентів;
- системне тестування призначене для перевірка повного ПЗ як єдиного цілого;

– приймальне тестування призначене для підтвердження готовності ПЗ до використання.

Приклад: У даному розділі розглянуто чотири основні рівні тестування, що забезпечують повноту перевірки ПЗ: модульне, інтеграційне, системне та приймальне. Такий підхід дає змогу поступово виявляти дефекти на різних рівнях абстракції та гарантувати відповідність кінцевого продукту технічним вимогам.

4. Очікувані результати тестування

Слід визначити, що тестування має надати такі результати: підтвердження коректності реалізації; оцінки надійності, стабільності, швидкодії; підготовку до приймання замовником.

Приклад: Результатом проведення тестування є підтвердження того, що ПЗ відповідає встановленим вимогам, функціонує стабільно та може бути передане для подальшої експлуатації.

Приклад підрозділу «Вступ» подано у додатку

Практична 1 «Планування приймального тестування»

Планування приймального тестування - частина процесу забезпечення якості ПЗ, яка визначає цілі, критерії, ресурси, відповідальність і послідовність дій, необхідних для підтвердження готовності ПЗ до впровадження.

UAT-тестування (User Acceptance Testing) або приймальне тестування не визначається єдиним стандартом, але його процеси описані у загальних стандартах тестування, таких як ISO/IEC/IEEE 29119 та IEEE 1012, які охоплюють весь життєвий цикл розроблення ПЗ. Ці стандарти надають керівництва для планування, виконання та документації процесів валідації та верифікації, до яких належить і UAT. Приймальне тестування дозволяє бути впевненим у валідності ПЗ до використання в умовах замовника, тому цей вид тестування називають ще валідаційнимтестуванням.

В основі тестування ПЗ лежить специфікація вимог до ПЗ (Software Requirements Specification, SRS) - формальний документ, який визначає призначення, функціональність, характеристики, обмеження та критерії приймання ПЗ. SRS описує що ПЗ має робити (а не як вона це робить) і є основою для розроблення, тестування та верифікації ПЗ.

Основна мета SRS - забезпечення єдиного розуміння вимог між замовником, розробником і тестувальником;створення основи для планування тестування (Test Plan, Traceability Matrix); мінімізація ризиків непорозумінь та зміни вимог на пізніх етапах.

Під час планування валідаційного тестування слід створити ряд документів, що дозволяє перевірити вимоги з SRS**:**

1. Функціональні вимоги*:* виконання ПЗ всіх заявлених функцій.

Наприклад: чи правильно обчислюються результати, чи працюють усі режими, чи доступні всі кнопки та меню.

2. Вимоги до інтерфейсу користувача**:** відповідність дизайну, зручність, зрозумілість повідомлень, коректність локалізації; відповідність зовнішнього вигляду технічному завданню.
3. Продуктивність і швидкодія**:** функціонування ПЗ у межах допустимих часових і ресурсних норм.
4. Сумісність: тестування роботи ПЗ на різних ОС, платформах, браузерах, пристроях; сумісність з апаратним забезпеченням або зовнішніми системами.
5. Надійність та відмовостійкість**:** реакцію ПЗ на збої, некоректні дані, відсутність з'єднання тощо; збереженість даних після збоїв.
6. Безпека*:* аутентифікацію, авторизацію, шифрування, захист від несанкціонованого доступу; відповідність політикам безпеки.
7. Вимоги до документації**:** наявність і коректність користувацьких інструкцій, технічних описів, довідкових файлів.
8. Вимоги до встановлення та експлуатації**:** правильність роботи інсталяторів; можливість оновленя /деінсталяції ПЗ без помилок.
9. Бізнес-вимоги (користувацькі сценарії): відповідність очікуваній поведінці ПЗ у реальних сценаріях використання.

Напр. "користувач може створити замовлення й отримати рахунок за 3 кроки".

«Планування приймального тестування» має визначати:

1. Мету приймального тестування - перевірку відповідності ПЗ встановленим вимогам замовника, специфікації вимог (SRS) та умовам договору.

Результати UAT мають визначати можливість прийняття ПЗ до експлуатації?

2. Об'єкт тестування

Назву ПЗ.

Версію, дату складання.

Короткий опис основних функцій та призначення ПЗ.

Платформу або середовище, у якому проводиться тестування.

3. Обсяг тестування

Які функції підлягають тестуванню.

Які не тестуються (наприклад, функції, не критичні для користувача).

Обмеження або передумови (наприклад, тестування лише основних сценаріїв користувача).

4. Критерії приймання

Критерії успішності: усі критичні та основні функції працюють відповідно до вимог; відсутні дефекти високої або середньої пріоритетності.

Критерії відхилення: наявність невиправлених критичних дефектів або невідповідність вимогам замовника.

5. Методике тестування

Типи тестів: функціональні, нефункціональні, користувацькі сценарії.

Використовувані методи: «чорна скринька», сценарне тестування, чеклісти.

Інструменти або середовища (якщо використовуються).

6. Тестові дані

Опис тестових сценаріїв і прикладів даних.

Джерела тестових даних (згенеровані, реальні, синтетичні).

7. Ролі та відповідальність (див у таблиці 1) Тут слід внести ПІБ замовника, розробника та тестувальника ПЗ

Таблиця 7 Перелік ролей та їх відповідальностей

ПІБ	Роль	Обов'язки
	Замовник / Користувач	Перевіряє відповідність очікуванням, затверджує результати
	Тестувальник	Проводить тестування, фіксує результати
	Розробник	Виправляє дефекти, консультує щодо реалізації функцій

8. Графік проведення тестування

Дати початку і завершення приймальних тестів.

Послідовність виконання сценаріїв.

Терміни подання звіту про результати.

9. Звітування та результати

Форма представлення результатів (таблиці з успішними/неуспішними тестами).

Оцінка відповідності вимогам.

Висновок про готовність ПЗ до впровадження.

10. Ризики та передумови

Можливі ризики (недоступність середовища, помилки у тестових даних).

Заходи мінімізації (резервний час, перевірка даних перед стартом).

11. Очікуваний результат

Після успішного завершення приймального тестування очікується підтвердження, що розроблене ПЗ повністю відповідає специфікації вимог і може бути впроваджене в експлуатацію.

Розглянемо пункт 6 «Тестові дані» більш детально:

Спочатку слід визначити вимоги до ПЗ (SWR)

Вимоги слід брати з курсового проєкту «Аналіз вимог до ПЗ», який Ви виконали восени: Функціональні вимоги (FRXX) – те, що має робити програма; Нефункціональні вимоги (NFRXX)– те, як повинна робити програма (надійність, безпечність, продуктивність тощо). Формуємо

таблицю *SRS (або використовуємо посилання на неї з першого розділу бакалаврської роботи)*:

Таблиця 8 Перелік вимог до ПЗ

Ідентифікатор вимоги	Визначення вимоги
FR01	ПЗ має приймати на вхід числа, що відповідають…
…	…
NFR01	ПЗ може бути використане тільки санкціонованим користувачем
…	…

Вимоги до ПЗ мають бути повними, однозначними, перевірними. Якщо ПЗ буде використовуватися у критичних галузях, то до вимог слід додати вимоги з відповідних нормативних документів.

Далі слід верифікувати вимоги до ПЗ , тобто перевірити:
чи немає суперечностей між вимогами ? чи можна протестувати кожну вимогу? чи всі вимоги простежуються? чи кожна з вимог має унікальний ідентифікатор?

Потім визначаємо стани ПЗ (таблицю зі станами ми також можемо знайти у 1-2 розділах бакалаврської роботи), наприклад:

1. ПЗ стартувало, але користувач в нього не зайшов
2. Користувач зайшов у програму і ввів пароль та логін
3. Перевірка даних користувача
4. Визначення мови діалогу (Українська або Англійська)
5. тощо

Далі слід побудувати власне тестові дані
Валідація ПЗ складається з виконання кількох тестових сценаріїв (*testScript*), які дозволяють перевірити виконання функціональних або нефункціональних вимог. Кожен сценарій складається з кількох тестів *(testCase)*. Визначимо рівень важливості (Severity 0..10**)** та пріоритет виконання (Priority 0…10) тестів. Це буде потрібно під час виконання валідаційного тестування.

Тестові сценарії описують загальну ідею перевірки або ситуацію, яку треба протестувати, але без детальної конкретизації кроків, очікуваних результатів і умов виконання.

Таблиця 9 - Відмінність між тестовим сценарієм і тестом

Ознака	Тестовий сценарій (Test Scenario)	Тест (Test Case)
Призначення	Визначає, *що потрібно перевірити*	Визначає, *як саме перевірити*
Рівень	Високий, концептуальний	Конкретний, покроковий

деталізації		
Описує	Функціональність або ситуацію (наприклад, «перевірити реакцію ПЗ на перегрів»)	Конкретні кроки дій, очікувані результати, умови, вхідні дані
Кількість	Зазвичай кілька десятків	Може бути сотні або тисячі
Приклад	«Перевірити стабільність температури при коливаннях датчика»	«1. Установити шум ±1.5°C; 2. Запустити симуляцію 60с; 3. Зафіксувати T(t); 4. Переконатися, що відхилення ≤2°C»

Сценарії — це *загальні ситуації тестування* (описані у Вашій таблиці).
Тести — це *реалізації сценаріїв* із конкретними параметрами, кроками й результатами.
Тестові сценарії — подаються у вигляді таблиці: вони описують логіку, що перевіряється.
Тести — деталізуються *для вибраних сценаріїв* (наприклад, TS-01 і TS-02), із конкретними даними, очікуваними та фактичними результатами.

Для проведення UAT ПЗ було розроблено набір тестових сценаріїв, що відображають типові та граничні умови експлуатації ПЗ. Мета кожного сценарію - перевірка відповідності фактичної поведінки ПЗ очікуваній, визначеній у специфікації вимог.

1. Вхідні передумови

ПЗ моніторингу та керування температурою (приклад: модель електричної праски з ПІД-регулятором).
Середовище виконання:

ОС: Linux Mint 22
Мова реалізації: C++17
Частота моделювання: 10 Гц
Інтерфейс введення: термодатчик (імітація через вхідний файл CSV або генератор сигналу у Xcos)

2. Тестові дані

Тут має бути опис тестів або сценаріїв, заснованих на SRS до ПЗ. Спрощено, для простого ПЗ, тестові дані можна подати так (табл.4)

Таблиця 10 Спрощені тестові дані для Приймального (Валідаційного) тестування (Acceptance Test Plan)

№	IDR	ID testScript	ID testCase	Test definition	State	Data	Expected		Severity	Priority
							Outputs	State		
1	NFR	NFR01.TS01	*NFR01.TS0*		1	Login=USER	OK	5	10	10

	01		*1. TC001*			Password=12346				
…										

У таблиці: IDR- унікальний ідентифікатор вимоги (див. табл. з переліком вимог) (NFR01 – перша нефункціональна вимога); IDtestScript – назва сценарію тестування, що може мати кілька тестів та використовується для перевірки вимоги до ПЗ (NFR01.TS01 – перший тестовий сценарій першої функціональної вимоги); IDtestCase – унікальний ідентифікатор тесту зі сценарію (NFR01.TS01. TC001 – перший валідаційний тест з першого сценарію першої нефункціональної вимоги); Test definition – визначення тесту; State – стан ПЗ перед виконанням тесту; Data – вхідні дані, що передаються у ПЗ; Expected outputs – очікуваний результат (повідомлення, або значення програмної змінної); Expected state – очікуваний стан після виконання тесту; Severity – важливість тесту; Priority- пріоритет тесту.

Для складного ПЗ таблицю з тестовими даними слід розбити на кілька таблиць (провести нормалізацію тестових даних), які визначають окремо тестові сценарії (TestScript) та власне тести (TestCase) (див Додаток А Планування приймального тестування).

У третьому розділі кваліфікаційній роботі слід показати тільки клька сценарів тестування та тестів, усі інші слід перенесту у додаток.

Усі вимоги до ПЗ слід перевірити на відповідність валідаційним тестам (тобто верифікувати). Для цього слід побудувати RTM (Requirement Traceability Matrix) матрицю, що визначає наявність тестів для кожної з вимог до ПЗ та інверсну до неї матрицю RTMI.

Таблиця 11 Валідаційна RTM матриця

SWR	Визначення SWR	Перелік валідаційних тестів

Таблиця 12 Валідаційна RTMI матриця

Валідаційний тест	Призначення валідаційного тесту	Перелік SWR

Ці таблиці слід розмістити у додатку до кваліфікаційної роботи бакалавра

У результаті ми зможемо сказати, що усі вимоги до ПЗ перевіряються валідаційними тестами і немає жодного валідаційного тесту, який би не перевіряв жодної вимоги до ПЗ.

У висновку підрозділа «Планування приймального тестування» слід:

1. Узагальнити мету та завдань приймального тестування
   коротко нагадати, що основною метою є перевірка відповідності розробленого ПЗ вимогам замовника та підтвердження готовності до впровадження;

зазначити, які саме аспекти (функціональні, нефункціональні, користувацькі) планується перевіряти.

2. Визначити результати планування приймального тестування
   описати, що було визначено у процесі планування:
   критерії прийняття (acceptance criteria);
   тестове середовище;
   ролі учасників (замовник, тестувальник, аналітик);
   перелік тестових сценаріїв і даних;
   графік або послідовність проведення тестів.
3. Показати очікувані результати тестування
   зазначити, що успішне проходження тестів засвідчить готовність продукту до експлуатації;
   окреслити очікувані показники якості (надійність, зручність, швидкодія тощо).
4. Зв'язати з наступними етапами життєвого циклу ПЗ
   зазначити, що результати приймального тестування є підставою для прийняття рішення про впровадження або доопрацювання;
   підкреслити, що воно завершує цикл тестування (після модульного, інтеграційного та системного).
5. Зробити короткий підсумковий висновок
   1–2 речення, які узагальнюють:
   "Таким чином, планування приймального тестування дозволило визначити основні критерії прийняття ПЗ, сформувати набір тестових сценаріїв і забезпечити готовність до проведення фінальної перевірки якості ПЗ."
   Приклад висновку є у Додатку А.

Практична робота 2 «Планування системного тестування »

Метою планування системного тестування є забезпечення організованого підходу до перевірки всього ПЗ, щоб підтвердити, що вона відповідає функціональним та нефункціональним вимогам замовника і готово до експлуатації. Планування системного тестування визначає обсяг робіт, ресурси, методи і критерії прийнятності результатів тестування.

Цілі планування системного тестування:

1. Визначення обсягу тестування – які саме функції, модулі, процеси та взаємодії у ПЗ підлягають перевірці.
2. Визначення типів тестування – функціональне, нефункціональне, регресійне, стрес-тестування, безпекове тестування тощо.
3. Вибір методів і підходів – інструментальні засоби, тестове середовище, щобудуть використовуватися під час тестування, ручне чи автоматизоване тестування, використання тестових сценаріїв і випадків.

4. Тестові дані для системного тестування
5. Планування ресурсів – призначення відповідальних, обладнання, середовищ тестування, програмних інструментів.
6. Встановлення критеріїв завершення тестування – умови, за яких ПЗ вважається протестованою та готовою до приймання.
7. Ідентифікація ризиків – визначення потенційних проблем, що можуть вплинути на якість тестування або на ПЗ вцілому.
8. Побудова графіку тестування – визначення термінів початку і завершення тестування, послідовності виконання сценаріїв.

Основні групи вимог, що перевіряються під час системного тестування ПЗ такі:

1. *Функціональні вимоги:* перевірка виконання ПЗ функцій, визначених у SRS: коректність реалізації бізнес-логіки; правильність взаємодії між модулями; відповідність вихідних результатів очікуваним; поведінка ПЗ при типових і граничних вхідних даних; перевірка коректності алгоритмів управління, оброблення даних, розрахунків.

Приклад: у ПЗ стабілізації дрона перевірка підтримки дроном заданої висоти протягом заданого часу при різних навантаженнях.

2. Нефункціональні вимоги **- п**еревірка якості функціонування ПЗ:

Продуктивності (Performance) ПЗ:
часу відгуку;
пропускної здатністі;
використання ресурсів (CPU, RAM, енергії);
затримки у каналах зв'язку.

Надійності (Reliability):
стійкості до збоїв, втрат сигналів;
відновлення після помилок;
кількості відмов за певний час.

Безпеки (Security) функціонування ПЗ:
автентифікації, авторизації, шифрування;
захисту від несанкціонованого доступу;
перевірка цілісності даних.

Зручності використання (Usability) ПЗ:
зручності інтерфейсу;
відповідності стандартам UX/UI;
легкості навчання користувача.

Сумісності (Compatibility) ПЗ:
роботи у різних середовищах (ОС, мережах, архітектурах);
інтеграції з іншими системами, драйверами, бібліотеками.

Масштабованості (Scalability) ПЗ: можливості розширення кількості користувачів, датчиків, пристроїв без деградації продуктивності.

Переносимості (Portability) ПЗ: можливості роботи ПЗ на різних апаратних або/та операційних.

3. Вимоги до конфігурації та середовища: коректність параметрів запуску, ініціалізації, логування; взаємодія з периферією або сенсорами; правильність налаштування комунікаційних інтерфейсів.

4. Вимоги до цілісності та відновлення даних: перевірка, що після перезавантаження або відмови ПЗ повертається до узгодженого стану; контроль журналів, кешів, буферів, баз даних.

Спочатку слід визначити системні вимоги до ПЗ
На підставі таблиці SRS слід кожній вимозі до ПЗ поставити у відповідність принаймі одну системну вимогу (*System Requirement*). Системні вимоги мають бути однозначними, зрозумілими і коректними до інших системних вимог. Ідентифікатори також мають бути унікальним, наприклад для FRXY ми визначили три системні вимоги, тоді їх ідентифікатори такі: FRXY. SR01, FRXY. SR02, FRXY. SR03.

Таблиця 13 Перелік Системних вимог (SR)

Ідентифікатор вимоги	Визначення системної вимоги
…	…
FRXY. SR01	ПЗ має формувати абсолютну похибку результату
…	…
NFRXY. SR01	ПЗ може бути використане тільки санкціонованим користувачем
…	…

Системні вимоги слід верифікувати, тобто перевірити на відповідність вимогам до ПЗ:

Таблиця 14 RTM (SWR, SR)

SWR	Визначення SWR	Перелік SR

Таблиця 15 RTM(SR,SWR)

SR	Визначення SR	Перелік SWR

У результаті ми зможемо сказати, що усі системні вимоги відповідають вимогам до ПЗ і у нас немає жодної системної вимоги, яка не витікає з вимог до програмного забезпечення.

Останні три таблиці слід розмістити у додатку до кваліфікаційної роботи

Для визначення тестових даних системного тестування слід використати детальний граф станів і переходів ПЗ. Під час системного тестування слід перевірити усі переходи між станами та подавати комбінації такі вхідних даних, що відповідають декартовому добутку їх класів еквівалентності.

Самі тестові дані для системного тестування мають структуру, подібну до тестових даних приймального тестування. У випадку простого ПЗ тестові дані можливо розмістити у одній таблиці, інакше слід розбити цю таблицю на кілька (дивись додаток А. Тестування. Планування системного тестування):

Таблиця 16 Прості тестові дані системного тестування

№	IDSR	ID testScript	ID testCase	Test definition	State	Data	Expected		Severity	Priority
							Outputs	State		
1	FR01.SR01	FR01. SR01.TS01	FR01. SR01.TS01.TC01		1	Login=USER Password=123456	OK	5	10	10
…										

Де: FR01.SR01 – перша системна вимога до першої функціональної вимоги; FR01. SR01.TS01 – перший тестовий сценарій для перевірки першої системної вимоги, що базується на першої функціональної вимоги FR01. SR01.TS01.TC01 – перший тест (test case), що входить у перший тестовий сценарій для перевірки першої системної вимоги

Усі системні вимоги слід перевірити на відповідність системним тестам (ST). Для цього слід заповнити дві матриці:

Таблиця 17 Системна RTM

SR	Перелік ST

Таблиця 18 Системна RTMI

ST	Перелік SR

У результаті ми зможемо сказати, що усі системні вимоги ми перевіряємо під час системного тестування та у нас не існує зайвих тестів, що не перевіряють жодної системної вимоги

Останні три таблиці слід розмістити у додатку до кваліфікаційної роботи

Далі слід зробити невеликий висновок про планування системного тестування:

1. Коротке резюме мети тестування

Зазначаються основні цілі системного тестування: перевірка відповідності ПЗ вимогам, забезпечення стабільності та безпеки системи, виявлення дефектів перед впровадженням.

2. Оцінка обсягу та охоплення тестування
   Які функціональні та нефункціональні вимоги будуть перевірятися.
   Пояснюється, які частини ПЗ охоплюються тестами.
3. Перелік основних тестових сценаріїв
   Коротко вказуються ключові типи тестів (функціональні, навантажувальні, безпекові тощо).
   Підкреслюється, що сценарії покривають критичні бізнес- та технічні процеси.
4. Очікувані результати планування
   Визначено критерії успішності тестів (наприклад, відповідність вимогам, відсутність критичних дефектів).
   Вказано, як результати тестування будуть використовуватися для прийняття ПЗ.
5. Висновки щодо готовності до виконання тестування
   Оцінка підготовки середовища тестування, наявності тестових даних, інструментів і ресурсів.
   Висновок про доцільність подальшого виконання системного тестування.
6. Рекомендації (за потреби)
   Можливі уточнення плану тестування, додаткові сценарії або підготовка даних.

Приклад висновку до планування системного тестування див. у додатку А. Тестування. Планування системного тестування

Практична робота 3 «Планування інтеграційного тестування»

Метою планування інтеграційного тестування є забезпечення узгодженості, сумісності та коректної взаємодії між окремими модулями (компонентами) ПЗ. На цьому етапі визначаються стратегії, обсяг, ресурси, графік і критерії успішності інтеграційного тестування, що дозволяє мінімізувати ризики помилок під час збирання ПЗ в єдине ціле.

Цілі планування інтеграційного тестування:

1. Визначення об’єкта тестування – модулі або інтерфейси, які підлягають інтеграції.
2. Визначення стратегії інтеграції – вибір підходу (наприклад, зверху-вниз, знизу-вгору, змішаний або інкрементальний).
3. Опис середовища тестування – інструменти, емулятори, драйвери, заглушки, платформи, необхідні для інтеграції.
4. Визначення критеріїв входу та виходу – умови, за яких модулі допускаються до інтеграції, і критерії завершення тестування.

5. Планування послідовності інтеграції – встановлення порядку об’єднання модулів для виявлення помилок взаємодії на ранніх етапах.
6. Підготовка тестових сценаріїв і даних – визначення типових і граничних випадків взаємодії між компонентами.
7. Оцінка ресурсів і часу – розрахунок необхідних людських, технічних і часових ресурсів для проведення тестування.
8. Планування контролю ризиків – визначення потенційних проблем інтеграції (несумісність інтерфейсів, передача даних, черги викликів тощо).
9. Встановлення критеріїв прийняття результатів – умови, за яких інтеграційне тестування вважається успішно завершеним.
10. Забезпечення трасування вимог – встановлення зв’язку між вимогами до ПЗ та тестовими сценаріями для підтвердження їх реалізації при інтеграції.

Основні вимоги, що перевіряються під час інтеграційного тестування:

1. Інтерфейсні вимоги

   Коректність обміну даними між модулями (формат, тип, порядок, обсяг).

   Перевірка правильності виклику API, методів, функцій, параметрів.

   Виявлення несумісності типів даних, некоректних конвертацій, порушення протоколів взаємодії.
2. Функціональні вимоги на рівні модулів

   Чи правильно реалізовано узгоджену логіку при спільній роботі кількох модулів.

   Наприклад: передача результатів обчислень з одного модуля до іншого, або ланцюжок “сенсор → контролер → виконавчий механізм”.
3. Вимоги до оброблення помилок

   Як ПЗ реагує на збої у суміжних модулях.

   Чи передаються винятки, чи є коректні повідомлення про помилки.

   Перевірка, що відмова одного компонента не спричиняє збоїв ПЗ.
4. Вимоги до послідовності викликів

   Перевіряється правильність порядку викликів методів або передачі даних.

   Наприклад, чи не відправляються запити до сервера до ініціалізації підключення.
5. Вимоги до продуктивності взаємодії

   Час відгуку при міжмодульній комунікації.

   Перевірка затримок і пропускної здатності каналів зв’язку.

6. Вимоги до сумісності між версіями компонентів

Узгодженість версій бібліотек, драйверів, API.

Відсутність конфліктів при оновленні окремих модулів.

7. Вимоги до узгодженості даних

Чи не втрачаються або не дублюються дані під час обміну між компонентами.

Перевірка цілісності, синхронності і правильності оновлення спільних структур даних або БД.

8. Вимоги до конфігурації та середовища

Коректність шляхів, змінних середовища, параметрів з'єднання.

Перевірка спільної роботи компонентів у різних конфігураціях (локально, через мережу, через шину даних тощо).

Завдання інтеграційного тестування полягає у перевірці взаємодії окремих модулів. План інтеграції та інтеграційного тестування має визначати архітектуру, модулі ПЗ (U) та порядок інтеграції модулів. Звичайно модуль є класом та має поля даних та кілька методів кожен слід описати окремо: призначення, формальні параметри, результат. На підставі розробленої архітектури ПЗ спочатку слід створити таблицю з переліком модулів та їх призначенням, а також системних вимог, що він реалізує та перелік системних тестів, що перевіряють системну вимогу:

Таблиця 19 Модулі ПЗ

Ідентифікатор модуля	Назва програмного модуля	Призначення модуля	Перелік системних вимог	Перелік системних тестів
U27	Avcd.cpp Avcd.hpp	Фільтрація даних	FR234.SR01	FR234. SR01.TS01.TC11 FR234.SR01.TS01.TC11
U28	DFGH.cpp DFGH.hpp	переднє оброблення даних	FR235.SR01	FR235. SR01.TS01.TC11

Далі слід для кожного з модулів визначити методи, їх призначення, вхідні дані та результати:

Таблиця 20 Методи ПЗ

Модуль	Назва метода	Призначення метода	Перелік вхідних даних	Перелік результатів
U27	LinearFiltr	Лінійна Фільтрація даних	A- масив double[10]	B- масив double [10]
	KalmanFilter	Фільтрація вхідних даних методом Кальмана	A- масив double[10]	B- масив double [10]

Спочатку слід визначити напрям інтеграції ПЗ: зверху-вниз, або знизу уверх, та які саме збірки модулів та у якому порядку будемо тестувати та які саме заглушки або драйвери будемо використовувати. Якщо ПЗ складається з чотирьох модулів і інтегрується послідовно, то цей процес може мати такий порядок:

Таблиця 21 – Порядок інтеграційного тестування

№	Збірка	Склад збірки (модулі та попередньо перевірені збірки)	Заглушки (Stub, Mock)
1	US01	U01, U02	U03Mock01, U04Mock01 U03Mock02, U04Mock02
2	US02	US01,U03	U04Mock03
3	US	US02, U04	відсутні

Нам потрібно п’ять заглушок, які будуть симулювати ще інтеграційно не перевірені модулі. Після виконання третього кроку ми матимемо повне ПЗ. Тепер можемо створити тестові дані для інтеграційного тестування. Для простого ПЗ тестові дані можливо подати у одній, що має структуру таблиці 19, а для складного потрібно створити кілька таблиць, що визначають тестові сценарії та тести (див. додаток А. Тестування. Планування інтеграційного тестування)

Таблиця 22 – Тестові дані Інтеграційного тестування

№	Збірка	Заглушки	IDtestscript	IDtestCase	Test definition	State	Data	Expected		Severity	Priority
								Outputs	State		
1	US01	U03Mock01, U04Mock01	TS01	TC01		1	--	Login: Password:	1	10	10
2	US01	U03Mock02, U04Mock02	TS01	TC02		1	Login=USER Password=123456	OK	5	10	9
3	US02	U04Mock03	TS02	TC03		2	x=1, y=2	Z=6	6	9	8
4	US0	---	TS03	TC04		3	f=1, g=2	R=10	7	9	8

	2										

Де Збірка – ідентифікатор збірки модулів, Заглушки – перелік заглушок, що використовуються для інтеграційного тестування збірки у поточному тесті, IDtestscript – тестовий сценарій для інтеграційного тестування збірки, IDtestCase – ідентифікатор інтеграційного тесту, Test definition – визначення тесту, State – стан ПЗ перед виконанням інтеграційного тесту, Data – вхідні дані тесту, Expected Outputs – очікуваний результат тесту, Expected State – очікуваний стан ПЗ після виконання інтеграційного тесту

Далі слід верифікувати модулі та інтеграційні тести (IT):

Таблиця 23 Модулі-тести

SWU	Перелік IT

Таблиця 24 Тести- модулі

IT	Перелік SWU

Ці таблиці дозволять визначити покриття програмних модулів інтеграційними тестами (Чи усі модулі були перевірені?) У результаті ми зможемо сказати, що усі програмні модулі ми перевіряємо під час інтеграційного тестування та у нас не існує зайвих інтеграційних тестів, що не перевіряють жодного модуля

Тепер верифікуємо методи модулів та тести:

Таблиця 25 Модулі-тести

Метод	Перелік IT

Таблиця 26 Тести- методи

IT	Перелік Методів

Ці таблиці дозволять визначити покриття методів програмних модулів інтеграційними системними тестами (Чи усі методи були перевірені?) У результаті ми зможемо сказати, що усі методи ми перевіряємо під час інтеграційного тестування та у нас не існує зайвих тестів, що не перевіряють жодного методу. Останні чотири таблиці слід перенести у додаток до кваліфікаційної роботи бакалавра

Приклад плану інтеграційного тестування див. Додаток В.

Висновок про планування інтеграційного тестування зазвичай включає:

1. Коротке резюме мети тестування, де пояснюється, що планування інтеграційного тестування спрямоване на перевірку взаємодії між модулями. Вказується, що тестування має забезпечити виявлення помилок інтеграції, несумісностей інтерфейсів та некоректної взаємодії модулів.

2. Оцінка покриття тестування визначає: які модулі будуть протестовані; наскільки плановані тести охоплюють критичні функціональні сценарії та взаємодії.
3. Визначення критеріїв успішності, які вказують, що інтеграційне тестування пройдено успішно (наприклад, всі інтерфейси функціонують коректно, передача даних між модулями без помилок).
4. Виявлені потенційні ризики та обмеження (наприклад, затримки у взаємодії модулів, обмеженість тестових даних, залежність від зовнішніх компонентів).
5. Рекомендації щодо подальших кроків: уточнення тестових сценаріїв, додаткові перевірки інтеграції або підготовка до системного тестування.
6. Загальний висновок, що визначає готовність плану інтеграційного тестування для реалізації, забезпечення систематичного підходу до перевірки взаємодії модулів ПЗ та готовність до планування модульного тестування .

   Приклад висновку до підрозділа дивись у додатку

Практична робота 4 «Планування модульного тестування»

Наступним етапом є розроблення «Плану модульного (автономного) тестування». Попередньо уже було сформовано список модулів та методів. Для кожного з методів слід визначити тестові вхідні дані і очікувані результати. Автономні тести для одного модуля слід оформити таблицею.

Автономне тестування (Unit Testing) — це найнижчий рівень тестування, під час якого перевіряють окремі модулі, функції або класи **ПЗ** ізольовано від решти.

Мета - впевненість, що кожен програмний модуль працює правильно та згідно зі своєю специфікацією та забезпечення готовності до проведення модульного тестування шляхом визначення обсягу, стратегії, ресурсів, критеріїв приймання та методів оцінювання якості окремих його модулів. Створення чіткої організаційної та технічної бази для своєчасного, повного та ефективного виконання тестів.

Цілі автономного тестування:

1. Визначення об'єктів тестування**:** встановлення того, які саме модулі (класи, функції, компоненти) підлягають тестуванню.
2. Вибір підходів і методів тестування*: о*брання відповідних методів тестування — структурне (white-box), функціональне (black-box) або комбіноване.
3. Розроблення тестових сценаріїв і даних*:* підготовка сценаріїв, тестів, вхідних даних та очікуваних результатів.
4. Визначення критеріїв початку і завершення тестування*:* встановлення умови, за яких тестування може бути розпочате, і коли його вважають завершеним.
5. Оцінка ресурсів і термінів: визначення необхідних ресурсів (людських, технічних, часових) для виконання тестів.
6. Визначення інструментів і середовища*:* обрання або налаштування інструментів автоматизації, бібліотек mock-об'єктів, середовищ виконання.
7. Призначення відповідальних осіб: розподіл ролей між учасниками тестування (розробник, тестувальник, аналітик).
8. Визначення ризиків і заходів мінімізації*: о*цінювання можливих ризиків під час тестування (недостатність даних, нестабільність коду тощо) та передбачити способи їх зниження

Вимоги, що перевіряються під час автономного тестування

1. Функціональна коректність модуля: чи виконує функція, клас або компонент свою основну логіку відповідно до технічного завдання; чи правильно обробляє типові, граничні та помилкові вхідні дані; чи формує очікувані вихідні результати.

2. Коректність оброблення вхідних**:** чи перевіряються діапазони вхідних параметрів; чи обробляються виняткові ситуації (division by zero, overflow); чи повертає функція коректні коди помилок або виключення (exceptions).

3. Вимоги до алгоритмічної точності**:** чи відповідають обчислення математичній моделі; перевірка похибок при обчисленнях з плаваючою комою (double, float); стабільність результатів при малих варіаціях вхідних даних.

4. Вимоги до внутрішніх структур і логіки**:** покриття всіх гілок (if, else, switch, loop); правильність умов переходів і циклів; коректність ініціалізації змінних і використання пам'яті.

5. Вимоги до інтерфейсів (API): чи повертає функція очікувані типи даних; чи дотримано протоколу виклику; чи не відбувається побічних ефектів (зміни зовнішніх змінних, станів, файлів).

6. Вимоги до продуктивності модуля**:** час виконання алгоритму; використання пам'яті; відсутність витоків пам'яті (особливо в C++).

7. Вимоги до тестового покриття:

statement coverage — чи виконано всі оператори;

branch coverage — чи протестовано всі розгалуження;

path coverage — чи охоплено всі можливі шляхи виконання.

Розроблений план модульного тестування (Unit Test Plan) має відповідати стандартам IEEE 829 та ISO/IEC/IEEE 29119:

ПЛАН МОДУЛЬНОГО ТЕСТУВАННЯ

1. Вступ

Мета документа

Об'єкт тестування (модулі, класи, функції)

Область застосування (що входить і не входить до тестування)

Посилання на супутні документи (технічне завдання, SRS, дизайн-документація, код, стандарти)

2. Об'єкти тестування

Перелік модулів, функцій або класів, що підлягають тестуванню

Опис взаємозв'язків між ними

Ідентифікатори або шляхи у репозиторії коду

3. Характеристика середовища тестування

Опис апаратного та програмного середовища

Використані компілятори, бібліотеки, інструменти тестування (наприклад, GoogleTest, Catch2, Boost.Test)

Параметри налаштування середовища (шляхи, змінні, макроси, конфігураційні файли)

4. Стратегія тестування

Обраний підхід (White-box, Black-box, Gray-box)

Методи тестування (unit-level validation, stubs, mocks, drivers)

Критерії покриття (лінійне, гілкове, шляхове, покриття умов)

Обґрунтування вибору підходів

5. Тестові сценарії та дані

Опис тестових випадків, що перевіряють основну, граничну, виняткову поведінку

Вхідні дані та очікувані результати

Формат представлення тестів (наприклад, у таблиці або у файлах тестів)

6. Критерії початку та завершення тестування

Умови, за яких тестування може бути розпочато (готовність коду, компіляція без помилок)

Критерії завершення (досягнуте покриття, усунення всіх критичних дефектів, стабільність результатів)

7. План виконання

Послідовність виконання тестів

Розподіл відповідальності між членами команди

Орієнтовні терміни виконання

8. Критерії оцінювання результатів

Правила інтерпретації результатів тестів (Pass/Fail)

Оцінка якості модулів на основі показників покриття, дефектів, стабільності

9. Ризики і способи їх мінімізації

Потенційні ризики: відсутність залежностей, нестабільні інтерфейси, неповне покриття

План дій для їх зменшення

10. Висновки

Очікувані результати тестування

Загальна готовність до проведення модульних тестів

Рекомендації щодо подальших етапів тестування (інтеграційного або системного)

Приклад плану модульного тестування є у додатку (Додаток А. Планування модульного тестування)

У спрощеному випадку тестові дані автонромного тестування можливо подати як одну таблицю (див. табл.24). У складних випадках слід створити кілька тадлиць, де визначити окремо тестогві сценарії та тести (див. додаток А. Тестування. Планування автономного тестування)

Таблиця 27 Тестові дані автономного тестування модуля U27

№	ID testScript	Метод	IDtestCase	Defi-ni-tion	Data	Expected Outputs	Severity	Priority

1	U27S01	LinearFiltr	U27S01TC01		A=2, B=7	R=8	10	9
		LinearFiltr	U27S01TC02		A=0, B=100	R=101	10	8
		LinearFiltr	U27S01TC03		A=-100, B=100	R=10000	10	7
2	U27S02	KalmanFilter	U27S02TC01		C=-100, D=0	Z=10	9	9
		KalmanFilter	U27S02TC02		C=-100, D=100	Z=1011	9	8

Далі слід верифікувати план автономного тестування, а саме перевірити, що ми автономно тестуємо усі модулі(класи) та усі методи. Ці таблиці слід розмістити у додатку кваліфікаційної роботи

Далі слід зробити невеликий висновок про планування модульного тестування, що має таку структуру:

1. Мета планування: Визначення об’єктів тестування (модулів, класів, функцій); формування стратегії проведення модульного тестування для перевірки коректності реалізації окремих компонентів ПЗ; забезпечення трасованості між вимогами, специфікацією і тестовими випадками.

2. Об’єкт тестування: назва програмного модуля або класу; короткий опис його функціонального призначення; вхідні та вихідні дані модуля; зовнішні залежності (інші модулі, бібліотеки, сенсори, інтерфейси тощо).

3. Підхід до тестування: методологія: *White-box / Black-box / Gray-box;* типи тестів: позитивні, негативні, граничні, виняткові ситуації; тестовий фреймворк (наприклад, Google Test, Boost.Test, Catch2 тощо); інструменти автоматизації, збірки, CI/CD; Критерії проходження тестів (успіх / помилка).

4. Структура тестів: опис модульних тестових функцій або сценаріїв; Таблиця “Вимога → Тест-кейс → Очікуваний результат”; формат іменування тестів (наприклад: Module_Function_Condition_ExpectedBehavior); підготовка тестового середовища (ініціалізація змінних, моків, емуляторів).

5. План покриття: які функції / класи будуть протестовані; очікуваний рівень покриття коду (%); план виконання тестів (черговість, залежності); оцінка ризиків (критичні частини коду, які потребують додаткових перевірок).

6. Критерії приймання результатів: Тест вважається пройденим, якщо…; тестовий набір вважається успішним при досягненні певного відсотка проходження; помилки реєструються у системі керування дефектами (наприклад, Jira, Redmine).

7. Висновки та рекомендації: узагальнення очікуваних результатів тестування; висновок про готовність до проведення модульних тестів;

перелік можливих ризиків і рекомендації щодо їх мінімізації; пропозиції з покращення архітектури тестів або модульної структури; висновок про готовність до виконання модульного тестування
Приклад висновку до підрозділа є у додатку

Практична робота 5 «Виконання модульного тестування»

Автономне або Модульне тестування (Unit testing) полягає у перевірці окремих функцій чи класів у відриві від решти ПЗ. Для цього існують такі інструменти: CppUTest [*1*], Jest [*2*], JUnit [*3*], NUnit [*4*] тощо.

Мета модульного (автономного) тестування: переконання, що кожен модуль працює ізольовано правильно, згідно з технічним завданням або специфікацією вимог до нього; забезпечення раннього виявлення помилок у логіці, обчисленнях або інтерфейсах; створення надійної основи для подальшого інтеграційного тестування.

Цілі модульного тестування ПЗ:

1. Перевірка коректності логіки модулів ПЗ (чи виконують модулі свої обчислення правильно?; чи правильно реалізовані умови, цикли, алгоритми?).
2. Перевірка відповідності модулів ПЗ вимогам до ПЗ (чи реалізують модулі усі функції, описані у специфікації? ;чи правильно обробляють модулі передбачені вхідні та вихідні дані?).
3. Виявлення локальних помилок у модулях ПЗ (синтаксичні або логічні помилки, які не видно при інтеграції; помилки в обробленні крайових або невалідних(некоректних) даних; помилки ініціалізації змінних, розрахунку формул тощо).
4. Перевірка оброблення виняткових ситуацій у модулях ПЗ (правильність реакції модулів на помилки введення/виведення, відсутність даних, переповнення буфера, тощо; правильність відновлення модулів після помилок, якщо це передбачено).
5. Перевірка інтерфейси модулів ПЗ (чи модуль правильно приймає параметри?; чи правильно повертає значення або повідомлення про помилки?).
6. Підвищення якості коду модулів ПЗ (виявлення і усунення дрібних дефектів на ранньому етапі тестування; спрощення подальшого налагодження ПЗ; покращення структури коду та зменшення його складності).
7. Підготовка ПЗ до інтеграційного тестування завдяки забезпеченню стабільної роботи модулів до подальшої інтеграції та зниженню ризиків каскадних помилок при подальшій інтеграції.

Приклади типових дефектів, які виявляються під час модульного тестування:

неправильна формула розрахунку (наприклад, переплутані знаки);
відсутність перевірки ділення на нуль;
некоректна ініціалізація змінних;
порушення меж масиву;
некоректне оброблення винятку або коду помилки;
функція повертає неправильне значення.

Для автономного тестування слід використати таблиці тестових даних, які було розроблено під час планування модульного тестування у які було додано кілька стопчиків:

Фактичні результати;

Executed - відмітка про виконання автономного тесту;

Pass/Fail - відмітка про успішність або провал тесту;

Comment – коментар стосовно виконання автономного тесту.

Далі треба сформувати звіт про виконання автономного тестування:

Структура Звіту про виконання модульного тестування

1. Загальні відомості: назва та версія ПЗ; дата проведення тестування; виконавець (ПІБ, група, посада); використане тестове середовище (операційна система, компілятор, бібліотеки, тестовий фреймворк).

2. Мета модульного тестування:

Коротко формулюється, що саме перевіряється і для чого:

верифікація окремих модулів;
перевірка правильності реалізації алгоритмів;
виявлення помилок на ранніх етапах.

3. Об’єкт тестування

Перелічуються модулі або класи, що були протестовані, наприклад:

SensorModule — оброблення даних з сенсора;
PIDController — розрахунок регуляторного впливу;
HeaterControl — формування сигналу керування нагрівачем.

4. Тестове середовище

Мова програмування (наприклад, C++17).
IDE / компілятор (наприклад, Visual Studio Code, GCC).
Тестовий фреймворк (наприклад, Google Test, Catch2).
Налаштування середовища, змінні середовища, шляхи до бібліотек.
Платформа виконання (Linux Mint / Windows / вбудована система).

5. Тестові сценарії (Test Cases)

Для кожного тестового сценарію зазначаються (див табл.22):

Таблиця 28 Тести тестового сценарію

№	Назва тесту	Вхідні дані	Очікуваний результат	Фактичний результат	Статус (Pass/Fail)

1	SensorRead_ValidInput	T=25°C	25 ±0.1°C	25.0°C	Pass
2	PID_OverTempResponse	Error=+20°C	Output ↓	Output ↓	Pass
3	Heater_ZeroPower	PowerCmd=0	Output=0	0	Pass

6. Результати тестування

Кількість виконаних тестів.

Кількість успішних / неуспішних.

Виявлені помилки (опис, причини, статус виправлення).

Оцінка покриття коду тестами (%).

Підсумкова таблиця результатів.

7. Висновки

Загальний стан якості модулів.

Оцінка відповідності вимогам.

Чи готові модулі до інтеграційного тестування.

Рекомендації (наприклад, повторне тестування після рефакторингу).

8. Додатки

Лістинги коду тестів.

Скриншоти результатів виконання тестів.

Автоматично згенеровані звіти (наприклад, gtest_output.xml або coverage.html).

Автономне тестування ПЗ слід проводити до тих пір поки усі помилки будуть виправлені, а усі тести виконані.

Висновок про автономне(модульне) тестування має бути таким:

1. Мета етапу

Коротко сформулювати, що модульне тестування виконується для перевірки коректності роботи окремих модулів (функцій, класів, компонентів).

Зазначити, що основна мета — виявлення помилок на ранніх етапах розроблення, коли виправлення дефектів є найменш витратним.

2. Об’єкт тестування

Вказати, які саме модулі або компоненти були протестовані. (Наприклад: модуль обчислення температурного режиму, модуль PID-регулятора, модуль оброблення даних сенсорів тощо.)

3. Виконані дії

описати, які тести проводилися:

позитивні та негативні сценарії;

граничні значення;

перевірка виключень (exception handling);

тестування інтерфейсів функцій.

зазначити, які інструменти або фреймворки використовувались (наприклад, Google Test, Boost.Test, JUnit, pytest тощо).

4. Отримані результати

узагальнити кількість успішних / неуспішних тестів;
відзначити, чи були знайдені помилки, і чи були вони усунені;
оцінити рівень покриття коду тестами (якщо визначався).

5. Оцінка якості модулів
зробити висновки про:
правильність логіки роботи модулів;
стабільність та відмовостійкість;
коректність взаємодії між модулями.

6. Загальний висновок і рекомендації
коротко підсумувати значення результатів: “Модульне тестування підтвердило працездатність базових компонентів ПЗ та правильність реалізації основних алгоритмів. Виявлені помилки були усунені, що підвищило надійність та готовність ПЗ до інтеграційного тестування.”
за потреби вказати рекомендації щодо подальшого вдосконалення коду або розширення тестів.

У додатку є приклад висновку з виконання модульного тестування

Практична робота 6 «Виконання інтеграційного тестування»

Інтеграційне тестування ПЗ слід розпочинати тільки після успішного завершення автономного тестування. Для спрощення інтеграційного тестування уснують такі інструменти: Cucumber [5], Junit[3]. Технології: SoapUI [6] та Postman [7].

Мета інтеграційного тестування - перевірка коректності взаємодії між окремими модулями або компонентами ПЗ після їх об’єднанняу. Тестування має підтвердити, що модулі правильно обмінюються даними, узгоджено виконують функції та не створюють конфліктів при спільній роботі.

Особлива увага приділяється виявленню дефектів на межах взаємодії, які не могли бути виявлені під час модульного тестування.

Цілі інтеграційного тестування

1. Перевірка інтерфейсів між модулями*:* переконанняя, що формати, типи даних, одиниці вимірювання та частоти обміну інформацією узгоджені між компонентами.
2. Виявлення помилок взаємодії*:* логічних, комунікаційних або синхронізаційних помилок, які проявляються лише під час обміну даними між модулями.
3. Оцінка узгодженості функціональності*:* переконання, що спільна робота модулів забезпечує правильне виконання функцій ПЗ згідно з вимогами до ПЗ (SRS).
4. Перевірка передачі управління і даних *:* забезпечення правильного виконання викликів функцій, оброблення повідомлень і розповсюдження сигналів між модулями.

5. Перевірка стійкості ПЗ при інтеграції: оцінювання поведінки ПЗ при зміні вхідних параметрів, появі помилкових або граничних сигналів.
6. Перевірка узгодженості фізичних параметрів:впевненість, передача фізичних величин (напруга, температура, момент, орієнтація тощо) відбувається в коректних одиницях та з потрібною точністю.
7. Підготовка до системного тестування *:* забезпечення стабільності роботи інтегрованого ПЗ, щоб подальше системне тестування було спрямоване на функціональну і нефункціональну валідацію.

   Результатом інтеграційного тестування є підтвердження того, що:

   усі модулі коректно взаємодіють між собою;
   обмін даними здійснюється без помилок і втрат;
   інтегроване ПЗ функціонує згідно з вимогами технічного завдання;
   ПЗ готове до системного тестування.

Приклади типових дефектів, які виявляються під час інтеграційного тестування:

- невідповідність форматів даних між модулями;
- втрата або дублювання повідомлень;
- неправильна послідовність ініціалізації;
- збої в обробці помилок;
- несумісність через різні версії API;
- проблеми з таймінгом у реальному часі (наприклад, затримки або блокування).

Для інтеграційного тестування за основу слід брати таблиці з плану інтеграційного тестування, до яких слід додати кілька стовпчиків:

1) Results - результати виконання інтеграційних тестів;
2) Execution - відмітка про виконання тесту;
3) Pass/Fail - відмітка про результат тесту;
4) Коментар

Приклад звіту з виконання інтеграційного тестування:

Метою виконання інтеграційного тестування була перевірка сумісної роботи програмних модулів після їх об’єднання, а також виявлення помилок взаємодії між компонентами, які не проявляються під час окремого (модульного) тестування. Основна увага приділялася правильності передавання даних між модулями, узгодженості інтерфейсів і відповідності отриманих результатів очікуваним.

Підготовка середовища

Інтеграційне тестування виконувалося в середовищі MS VS з використанням інструментальних програмних засобів.

Для забезпечення фізичної достовірності модель містила:

модуль сенсора (вимірює температуру або кутову швидкість);
модуль регулятора (ПІД або орієнтаційний);
модуль виконавчого пристрою (нагрівач, двигун, актуатор);
модуль комунікації (обмін сигналами між блоками);
інтерфейс моніторингу (виведення результатів симуляції).

Використовувались реалістичні параметри моделі, включаючи часові константи, коефіцієнти підсилення, інерційність виконавчих елементів, шум сенсора та обмеження вихідного сигналу.

Послідовність дій

Інтеграційне тестування проводилося поетапно:

1. Ініціалізація компонентів**.** Перевірено, що всі модулі правильно ініціалізуються, мають узгоджені параметри й типи даних.
2. Інтеграція модулів у єдине ПЗ. Виконано з'єднання сигналів між модулями згідно з архітектурною схемою.
3. Перевірка комунікацій. Оцінено правильність обміну даними між блоками: формати, частота, одиниці вимірювання.
4. Запуск тестових сценаріїв**.** Випробувано ПЗ при нормальних, перехідних та граничних режимах роботи.
5. Фіксація результатів**.** Отримані результати записано у таблиці тестових сценаріїв і графічно проаналізовано.

Таблиця 29 Тестові сценарії інтеграційног тестування

№	Назва сценарію	Вхідні дані	Очікуваний результат	Результат
1	Ініціалізація ПЗ	Пуск модулів з початковими параметрами	Усі модулі переходять у стан «готовий»	Пройдено
2	Номінальний режим	Задана уставка 50°C	Температура стабілізується ±1°C	Пройдено
3	Різка зміна уставки	Стрибок 50°C → 70°C	Без перерегулювання понад 5%	Пройдено
4	Збій сенсора	Втрата даних 0,5 с	ПЗ переходить у безпечний режим	Пройдено
5	Обмеження виконавця	Максимальна потужність 100%	ПЗ відновлює стабільність після насичення	Пройдено

Отримані результати

Під час виконання інтеграційного тестування було встановлено, що:

обмін сигналами між модулями відбувається без втрат даних;
затримка в комунікації не перевищує допустимі 10 мс;
регулятор правильно реагує на зміни сигналів сенсора;
ПЗ зберігає стабільність при зміні уставки;
у разі відмови сенсора активується режим захисту.

Графіки температури та вихідної потужності підтверджують правильність функціонування зворотного зв'язку. Відхилення не перевищують проєктних допусків.

Виявлені дефекти

Під час тестування зафіксовано один незначний дефект: при зміні частоти оновлення сигналу сенсора до 1 кГц спостерігалася затримка реакції регулятора (~20 мс).

Дефект усунуто зменшенням частоти вибірки до 200 Гц і корекцією коефіцієнтів ПІД-регулятора.

Аналіз результатів

Результати підтвердили правильність інтеграції модулів та узгодженість фізичних розмірностей у передачі сигналів.

ПЗ демонструє стійку роботу в номінальних та перехідних режимах.

Таблиця 30 – Результат інтеграційне тестування

Test Execution Report Template						
Назва збірки	Загальна кількість написаніх тестів/ Totall N of T.C written	Загальна кількість виконаних тестів / Totall N of T.C execution	Загальна кількість коректно виконаних тестів / Totall N of T.C Pass	Загальна кількість провалених тестів /Totall N of T.C Fail	Відсоток коректних тестів/ Pass%	Відсоток провалених тестів/ Fail%
Збірка 1	170	170	168	2	98%	2%
Збірка 2	90	90	88	2	97%	3%
Повне ПЗ	127	127	127	0	100%	0%
Total	387	387	383	4	98%	2%

Отримані результати відповідають очікуваним показникам точності, стабільності та швидкодії.

Висновок

Інтеграційне тестування виконано успішно. Усі тести завершились позитивно, а ПЗ виявило здатність до коректного функціонування при взаємодії окремих модулів. Виявлені незначні відхилення усунуті. ПЗ готове до системного тестування**.**

Практична робота 7 «Виконання системного тестування»

Системне тестування — це етап перевірки інтегрованого ПЗ для оцінки його відповідності визначеним вимогам (як функціональним, так і нефункціональним).

Мета системного тестування*:* перевірка відповідності ПЗ вимогам специфікації, виявлення помилок інтеграції, конфігурації, продуктивності, підтвердження готовності ПЗ до приймально-здавального тестування (Acceptance Testing) або як це інакше називають валідаційного тестування. Це останній етап перед приймально-здавальним тестуванням.

Цілі системного тестування:

1. Перевірка відповідності ПЗ специфікації вимог (SRS) (чи реалізовані всі функції, передбачені вимогами замовника?).
2. Виявлення дефектів, що виникли після інтеграції компонентів (перевірка коректності взаємодія між модулями наприклад, між модулями сенсорів, контролера та приводу).
3. Перевірка поведінки ПЗ в типових і граничних умовах (наприклад, при пікових навантаженнях, відмовах обладнання або екстремальних вхідних даних.
4. Оцінка нефункціональних характеристик ПЗ (надійність, швидкодія, масштабованість, безпека, зручність користування тощо).
5. Підтвердження готовності ПЗ до приймально-здавальних випробувань (Acceptance Testing)

Системне тестування є фінальним етапом внутрішньої перевірки перед передачею продукту замовнику. Системне тестування виконується після закінчення інтеграційного тестування та полягає у перевірці системних вимог. Через велику кількість системних тестів часто виконується автоматизовано за допомогою інструментів автоматизованого тестування, наприклад: Appium [8], Cypress [9], Playwright [10],Selenium [11], mabl [12], Taurus [13]. Тестування продуктивності (навантажувальне тестування) легко здійснюється зінструментами: Locust [14], що дозволяє створювати та виконувати навантажувальні тести; Apache Bench (AB) [15], що дозволяє автоматизувати навантажувальне тестування веб-серверів. Візуальне тестування та моніторинг може виконуватися інструментом: Applitools [16], що використовує штучний інтелект для автоматизації візуального тестування та моніторингу якості додатків.

За основу таблиці виконання системного тестування слід обрати тестові дані, зщо були розроблені під час «Планування системного тестування», до яких слід додати кілька стовпчиків:

1) Results - результати виконання системного тесту;
2) Execution - відмітка про виконання тесту;
3) Pass/Fail - відмітка про результат тесту;
4) Тип помилки: Defect, Bug,Error, Issue, Mistake,Failure
5) коментарем, що визначає подальші дії з ПЗ та знайденою помилкою (статус помилки): new/open, assigned, fix, re-open, closed

Існують такі типи програмних помилок:

Таблиця 31 - Типи програмних помилок

Термін	Опис/Description	Хто формує/ Raised by
Defect	ПЗ не працює так, як визначено вимогами	тестувальник
Bug	Неформальна назви дефекту	Тестувальник
Error	Проблема винакає у програмному коді	Розробник, Інженер з автоматизації тестування

Issue	ПЗ не відповідає бізнес-вимогам	Замовник
Mistake	Проблема у помилковій документації до ПЗ	--
Failure	Існування багатьох дефектів призводить до збою ПЗ	--

В результаті системного тестування мають бути розроблені такі документи:

Звіт про системне тестування ПЗ з приблизно такою структурою:

1. Загальні відомості: назва проєкту: ПЗ стабілізації CubeSat; версія 1.0; дата тестування: 24.10.2025; Тестувальник: [Прізвище, Ім'я]; мета тестування: Перевірка відповідності ПЗ вимогам специфікації (SRS)

2. Об'єкт тестування

Тип системи: ПЗ керування орієнтацією супутника

Компоненти:

Модуль сенсорів (гироскоп, магнітометр, сонячний сенсор)

Контролер (ПІД-регулятор)

Виконавчі механізми (реакційні колеса, магнітні котушки)

Вхідні дані: кутові швидкості, орієнтаційні кути, момент інерції

Вихідні дані: керуючі моменти, сигнали PWM

3. Перевірені під час системного тестування вимоги

Таблиця 32 - Перелік перевірених під час системного тестування вимог

№	Тип вимоги	Опис вимоги	Очікуваний результат	Статус
1	Функціональна	ПЗ стабілізує орієнтацію за 10 с	Орієнтація стабільна	Пройдено
2	Функціональна	Корекція при відхиленні > 5°	Повернення у межі ±1°	Пройдено
3	Нефункціональна	Споживана потужність < 5 Вт	4,6 Вт	Пройдено
4	Нефункціональна	Час реакції контролера ≤ 0.2 с	0.15 с	Пройдено
5	Надійність	Без відмов протягом 100 симуляцій	100/100 успішно	Пройдено

4. Перевірені під час системного тестування сценарії

Таблиця 33 - Перелік перевірених тестових сценаріїв

№	Сценарій	Вхідні умови	Очікувана поведінка	Результат
1	Запуск ПЗ у номінальному режимі	t=0, θ=10°, ω=0.1 рад/с	Стабілізація за <10 с	ОК
2	Перевірка граничних кутів	θ=45°	Плавне повернення без осциляцій	⚠ Мінімальні коливання
3	Збій сенсора	Gyro fault = 1	Аварійний перехід у безпечний режим	ОК

5. Результати виконання системних тестів: загальна кількість тестів - 1500; пройдено успішно - 1495; з попередженнями – 5; критичних збоїв – 0. Покриття вимог: 100% функціональних, 80% нефункціональних

6. Аналіз результатів: ПЗ задовольняє усім вимогам SRS; виявлено незначну чутливість регулятора до різких відхилень; потрібна додаткова перевірка при зміні моменту інерції.

7. Висновки: системне тестування ПЗ завершено успішно. ПЗ готове до приймально-здавальних випробувань. Рекомендації: Додатково проаналізувати стабільності ПЗ при змінних параметрах та перевірити поведінку ПЗ при затримках зв'язку.

Після виконання системного тестування слід сформувати таку таблицю

Таблиця 34– Результати системного тестування

Test Execution Report Template					
Загальна кількість написаніх тестів/ Totall N of T.C written	Загальна кількість виконаних тестів / Totall N of T.C execution	Загальна кількість коректно виконаних тестів / Totall N of T.C Pass	Загальна кількість провалених тестів /Totall N of T.C Fail	Відсоток коректних тестів/ Pass%	Відсоток провалених тестів/ Fail%
1500	1497	1492	5	99.7%	0.3%

Системне тестування слід проводити до тих пір поки усі помилки будуть виправлені

Тепер слід зробити висновок про системне тестування ПЗ:
Висновок про виконання системного тестування повинен мати приблизно таку структуру

1. Підсумок мети системного тестування: коротко нагадати, що системне тестування спрямоване на перевірку повної інтеграції ПЗ як єдиного цілого, з метою визначення його відповідності функціональним та нефункціональним вимогам; підкреслити, що цей етап перевіряє взаємодію всіх модулів і компонентів.

2. Опис виконаних дій: зазначити, які типи тестів проводились (функціональні; продуктивності (performance); безпеки (security); сумісності (compatibility); зручності використання (usability) тощо);

описати, у якому середовищі виконувались тести (конфігурація, ПЗ, апаратне забезпечення); вказати, які інструменти або засоби тестування застосовувалися.

3. Отримані результати

узагальнити успішність проходження тестів;

зазначити, які дефекти були виявлені та виправлені;

оцінити відповідність ПЗ вимогам (повністю / частково / із зауваженнями).

4. Оцінка якості ПЗ - коротко проаналізувати, як результати системного тестування вплинули на: надійність; стабільність роботи; масштабованість; відмовостійкість; відповідність нефункціональним вимогам (продуктивність, безпеку, UX тощо).

5. Висновки та рекомендації: зробити узагальнюючий висновок:"Результати системного тестування підтвердили стабільність роботи ПЗ в цілому та його відповідність визначеним вимогам. Виявлені незначні недоліки не впливають на основну функціональність і можуть бути усунені на етапі підготовки до приймального тестування."

За потреби зазначити рекомендації щодо подальшої оптимізації або доопрацювання, а також щодо переходу до наступного етапу — приймального тестування.

Приклад висновку дивися у додатку

Практична робота 8 «Виконання приймального тестування»

Слід сказати, що приймальне (валідаційне) тестування можливе тільки тоді, коли усі помилки виправлені і ПЗ перевірено від час системного тестування.

Мета приймального тестування (Acceptance Testing Purpose) - підтвердження того, що ПЗ відповідає вимогам замовника, зазначеним у технічному завданні (SRS або TOR), і є готовим до впровадження або передавання в експлуатацію. Іншими словами, це фінальна перевірка, яка визначає, чи приймає замовник розроблене ПЗ як придатне для використання за призначенням.

Цілі приймального тестування (Acceptance Testing Objectives) ПЗ

1. Перевірка відповідності ПЗ вимогам замовника – підвердження того, що всі функціональні та нефункціональні вимоги, визначені в SRS, реалізовані правильно.
2. Оцінка готовності ПЗ до експлуатації **– п**еревірка того, що ПЗ може працювати стабільно в реальному середовищі з очікуваними навантаженнями, даними й користувачами.
3. Перевірка бізнес-логіки та користувацьких сценаріїв – переконання у тому, що ПЗ задовольняє практичні потреби кінцевих користувачів і підтримує типові робочі процеси.
4. Оцінка якості та надійності ПЗ - перевірка відсутності критичних дефектів, що можуть перешкоджати нормальній роботі.
5. Перевірка інтеграції із зовнішніми системами – підтвердження правильності обміну даними, взаємодії з обладнанням та іншими системами.

6. Прийняття рішення щодо впровадження на основі результатів тестування, коли замовник (або приймальна комісія) ухвалює рішення:

або прийняти ПЗ;

або надати перелік зауважень для доопрацювання;

або відхилити продукт як невідповідний вимогам.

Результатом приймального тестування є: Акт приймання-здачі робіт.

У разі успіху — ПЗ приймається в експлуатацію. Якщо виявлено критичні невідповідності, то ПЗповертається на доопрацювання

У цьому пункті визначається, що приймальне тестування слід виконувати у відповідності зі стратегією та за тестовими даними приймального тестування, що визначені у підрозділі «Планування приймального тестування». Результати тестуванння показати у таблиці, що має таку структуру (див. таблицю)

Таблиця 35 - Приклади тестів для ПЗ керування температурою

ID тесту	Сценарій	Мета тесту	Вхідні дані / Умови	Очікуваний результат	Фактичний результат	Статус
TC-01.01	TS-01 – Початковий нагрів	Перевірити, чи досягає температура заданого значення (180°C) за ≤60 с	Tinit = 25°C, Tset = 180°C, P_max = 1000 Вт	Температура зростає до 180 ± 2°C не пізніше ніж через 60 с	179.6°C за 54с	Pass
TC-02.01	TS-02 – Підтримання температури	Перевірити стабільність після нагріву при сталому навантаженні	Tset = 180°C, стабільна кімнатна температура	Відхилення ≤ ±2°C протягом 5 хвилин	ΔT = ±1.3°C	Pass
TC-03.01	TS-03 – Реакція на зміну температури навколишнього середовища	Перевірити реакцію PID на падіння температури середовища з 25°C до 15°C	Tset = 180°C, зміна середовища на 15°C	Перехідний процес завершується ≤ 30 с, температура стабілізується 178–182°C	Відновлення за 26 с, T = 179.4°C	Pass
TC-04.01	TS-04 – Аварійне перевищення температури	Перевірити, чи ПЗ вимикає підігрів при T > 200°C	Імітація несправності сенсора або PID	Нагрів вимикається, подається сигнал «Overheat»	Підігрів вимкнувся при 201.5°C, подано сигнал	Pass
TC-05.01	TS-05 – Зниження напруги живлення	Перевірити, як змінюється швидкість нагріву при U = 180 В (замість 220В)	Tset = 180°C, U = 180 В, P_max = 0.67 номіналу	Час нагріву збільшується пропорційно зменшенню потужності, без коливань температури	Нагрів до 180°C за 83 с, стабільність збережено	Partial Pass

Пояснення до таблиці:

TC-xx.yy — ідентифікатор тесту (де xx відповідає номеру сценарію, yy — номеру конкретного тесту).

Очікуваний результат — критерій успішності (виходить із вимог або специфікації SRS).

Фактичний результат — отриманий під час симуляції або експерименту.

Статус:

Pass — тест пройдено успішно;

Partial Pass — виконано частково (може вимагати уточнення PID-параметрів);

Fail — тест не пройдено.

У третьому розділі кваліфікаційній роботі слід показати не більше однієї сторінки таблиць виконання валідаційного тестування. Усі інші таблиці виконаних тестів слід перенести у Додаток кваліфікаційної роботи бакалавра.

Звіт про приймальне тестування (Acceptance Test Report, ATR) - офіційний документ, який підтверджує, що ПЗ пройшло перевірку відповідності вимогам і може бути прийняте замовником або передане в експлуатацію.

Нижче наведено типовий шаблон і приклад структури звіту:

1. Загальні відомості

Назва продукту: *ПЗ стабілізації висоти дрона*

Версія ПЗ**:** v1.2.3

Дата проведення тестування**:** 20–23 жовтня 2025

Місце проведення**:** Лабораторія 342

Відповідальні особи:

Тестувальник: Іваненко І.І.

Розробник: Петренко П.П.

Замовник: ТОВ «AeroSpace»

2. Мета приймального тестування – підтвердження того, що ПЗ відповідає функціональним і нефункціональним вимогам, визначеним у Специфікації вимог (SRS), і може бути прийняте замовником для промислової експлуатації.

3. Об’єкт тестування - *п*рограмно-апаратний комплекс, що виконує стабілізацію висоти дрона в реальному часі, включаючи:

модуль сенсорного зчитування (ультразвуковий або барометричний датчик);

контролер (PID-регулятор);

приводи двигунів (через ESC);

інтерфейс телеметрії та візуалізації.

4. Критерії приймання: ПЗ вважається прийнятим, якщо:

1. Усі функціональні вимоги виконано.
2. Нефункціональні вимоги (швидкодія, стабільність, точність, безпека) задоволені.
3. Критичних або блокуючих дефектів не виявлено.

4. Рівень успішного проходження тестів ≥ 98%.
5. Перелік проведених приймальних тестів (таблиця)

Таблиця 36 - Перелік проведених приймальних тестів

№	**Назва приймального тесту**	**Тип тесту**	**Очікуваний результат**	**Фактичний результат**	**Статус**
1	Перевірка стабілізації при зміні навантаження	Функціональний	Висота зберігається ±5 см	Відповідає	Пройдено
2	Реакція на втрату сенсорних даних	Надійність	Перехід у безпечний режим	Відповідає	Пройдено
3	Тест продуктивності при 10 Гц оновлення	Продуктивність	Реакція ≤ 50 мс	Відповідає	Пройдено
4	Випробування з перебоями живлення	Стійкість	Відновлення стану після перезапуску	Частково відповідає	Уточнити

6. Результати тестування подано у таблиці:

Таблиця 37 - Результати валідаційного тестування

Загальна кількість написаних тестів/ Totall N of T.C written	Загальна кількість виконаних тестів/ Totall N of T.C execution	Загальна кількість пройдених тестів/ Totall N of T.C Pass	Загальна кількість провалених тестів/ Totall N of T.C Fail	Відсоток пройдених тестів Pass%	Відсоток провалених тестів Fail%
100	100	99	1	99%	1%

7. Виявлені дефекти

Таблиця 38 – Дефекти, які було виявлено під час валідаційного тестування

№	Опис дефекту	Критичність	Статус	Коментар
1	Тимчасова затримка при відновленні живлення	Середня	В процесі усунення	Не впливає на основну функціональність

8. Висновки

Результати приймального тестування підтверджують, що ПЗ відповідає вимогам SRS, стабільно функціонує в робочих режимах, готове до дослідної експлуатації.

Рекомендація: допустити ПЗ до промислової експлуатації після усунення дрібних недоліків.

9. Підписи сторін

Посада	ПІБ	Підпис	Дата
Тестувальник	Іваненко І.І.	_____	_____
Розробник	Петренко П.П.	_____	_____
Замовник	Коваль	_____	_____

	О.О.		

Висновок курсового проєкту

Висновок до курсового проєкту має бути таким, щоб його можливо було використати як висновок до третього розділу кваліфікаційної роботи бакалавра.

Загальний висновок про тестування ПЗ повинен мати таку структуру:

Висновок про тестування ПЗ

У результаті проведення тестування ПЗ було здійснено повну перевірку його функціональності, надійності, продуктивності та відповідності вимогам, визначеним у специфікації вимог (SRS).

Під час тестування проведено:

Модульне тестування — перевірено коректність роботи окремих модулів, відсутність логічних помилок та коректність оброблення вхідних даних.

Інтеграційне тестування — оцінено взаємодію між модулями, перевірено коректність обміну даними та узгодженість інтерфейсів.

Системне тестування — перевірено роботу ПЗ в цілому відповідно до функціональних і нефункціональних вимог.

Приймальне тестування — підтверджено готовність продукту до впровадження у промислову (експлуатаційну) середу.

Результати тестування показали, що:

усі функціональні вимоги виконані;

нефункціональні характеристики (продуктивність, стабільність, безпека, зручність використання тощо) перебувають у допустимих межах;

критичних дефектів не виявлено;

ПЗ відповідає критеріям приймання та готове до експлуатації.

Таким чином, проведене тестування підтвердило, що розроблене ПЗ є функціонально коректним, стабільним у роботі та відповідає вимогам замовника**.** Подальші роботи можуть бути спрямовані на оптимізацію продуктивності, підвищення масштабованості та удосконалення інтерфейсу користувача.

Третій розділ кваліфікаційної роботи бакалавра має містити не менше 10 посилань на літературу, оформлених за допомогою vak.in.ua

Усі таблиці, що неможливо додати через обмеження загального обсягу розділу, слід перенести у Додаток до кваліфікаційної роботи бакалавра, але треба посилатися на них у тексті третього розділу.

ПІСЛЯМОВА

Завершення роботи з даним посібником та виконання курсового проєкту з дисципліни **«Тестування та верифікація програмного забезпечення»** є важливим етапом у професійному становленні здобувачів вищої освіти в галузі інформаційних технологій. У процесі навчання студенти не лише опанували теоретичні основи тестування та верифікації ПЗ, але й набули практичних навичок, необхідних для реальної інженерної діяльності.

Під час виконання курсового проєкту здобувачі навчилися формалізувати вимоги до програмних систем, обґрунтовано обирати рівні та методи тестування, розробляти повноцінну тестову документацію, планувати та виконувати різні види тестування, аналізувати результати та приймати інженерно виважені рішення щодо якості ПЗ. Особливу цінність має те, що ці знання та вміння були сформовані на прикладі власних програмних розробок, що максимально наближує навчальний процес до умов реальної професійної практики.

У ході роботи студенти отримали практичний досвід використання сучасних інструментів тестування, засобів автоматизації, систем керування тестовою документацією та дефектами, а також навчилися працювати з технічною та нормативною документацією. Це дозволило сформувати цілісне уявлення про роль тестування та верифікації в життєвому циклі ПЗ та їхній вплив на надійність, безпеку й конкурентоспроможність програмних продуктів.

Набуті в межах дисципліни знання та навички створюють міцне підґрунтя для подальшої професійної діяльності випускників. Вони дозволяють упевнено виконувати широкий спектр завдань у галузі інформаційних технологій — від аналізу вимог і розроблення ПЗ до його тестування, верифікації, супроводу та вдосконалення. Сформовані компетентності забезпечують готовність до роботи як у командних проєктах, так і до самостійного вирішення складних інженерних задач у сучасному ІТ-середовищі.

Таким чином, опанування матеріалу даного посібника та успішне виконання курсового проєкту засвідчують, що здобувачі зробили вагомий крок до становлення кваліфікованими фахівцями, здатними відповідально та професійно працювати з програмними системами будь-якого рівня складності.

ДОДАТОК А.
Тестування ПЗ кваліфікаційної роботи бакалавра

А.1 Приклад вступу до третього розділу бакалаврської роботи

Тестування ПЗ є завершальним і одним із найважливіших етапів життєвого циклу розроблення. Його мета полягає у виявленні помилок, перевірці правильності реалізації функцій та оцінюванні відповідності створеного ПЗ вимогам замовника.

У процесі тестування перевіряються не лише функціональні характеристики, але й надійність, ефективність, зручність використання та стабільність роботи програми.

Для повноти перевірок ПЗ застосовується багаторівневий підхід, що включає модульне, інтеграційне, системне та приймальне тестування. Кожен із цих рівнів має власну мету, об’єкти перевірки та очікувані результати.

У даному розділі розглянуто методику проведення тестування на кожному рівні, приклади тестових сценаріїв і результати їх виконання, що підтверджують працездатність і готовність ПЗ до впровадження.

А.2 Планування приймального тестування

А.2.1 Приклад System Requirements Specification (SRS)

System Requirements Specification (SRS) для інформаційної системи управління бібліотекою

1. Вступ

1.1 Призначення документа

Цей документ визначає системні вимоги до **інформаційної системи управління бібліотекою** (далі — Система). SRS призначений для розробників, тестувальників, замовників та інших зацікавлених сторін і використовується як базовий документ для верифікації та валідації Системи.

1.2 Область застосування

Система призначена для автоматизації процесів обліку книжкового фонду, реєстрації читачів, видачі та повернення книг, а також формування звітів.

1.3 Визначення, скорочення

- **SRS** — System Requirements Specification
- **UI** — User Interface
- **DB** — Database

2. Загальний опис

2.1 Перспектива продукту

Система є веборієнтованим програмним продуктом, що працює за клієнт-серверною архітектурою та використовує централізовану базу даних.

2.2 Класи користувачів

- **Адміністратор**
- **Бібліотекар**
- **Читач**

2.3 Обмеження

- Система повинна працювати у сучасних веббраузерах.
- Мова інтерфейсу — українська.
- База даних — реляційна.

3. Функціональні вимоги

FR-1 Реєстрація користувачів

Система повинна забезпечувати реєстрацію нових читачів із збереженням персональних даних у базі даних.

FR-2 Аутентифікація користувачів

Система повинна забезпечувати вхід користувачів за логіном і паролем.

FR-3 Управління книжковим фондом

Система повинна дозволяти бібліотекарю:

- додавати книги;
- редагувати інформацію про книги;
- видаляти книги.

FR-4 Видача та повернення книг

Система повинна реєструвати операції видачі та повернення книг із фіксацією дати та користувача.

FR-5 Пошук книг

Система повинна забезпечувати пошук книг за назвою, автором та ISBN.

4. Нефункціональні вимоги

NFR-1 Продуктивність

Система повинна обробляти не менше 100 одночасних користувачів без деградації продуктивності.

(ISO/IEC 25010 — Performance efficiency)

NFR-2 Надійність

Система повинна забезпечувати доступність не менше 99 % у робочий час.

(ISO/IEC 25010 — Reliability)

NFR-3 Зручність використання

Новий користувач повинен мати змогу виконати базові операції без попереднього навчання.

(ISO/IEC 25010 — Usability)

NFR-4 Безпека

Система повинна забезпечувати:

- захист персональних даних;

- розмежування прав доступу.
  (ISO/IEC 25010 — Security)
  NFR-5 Супроводжуваність

Архітектура Системи повинна дозволяти модифікацію функціоналу без зміни існуючих модулів.
(ISO/IEC 25010 — Maintainability)

5. Інтерфейсні вимоги

5.1 Користувацький інтерфейс

- вебінтерфейс;
- адаптивний дизайн;
- підтримка клавіатурної навігації.

5.2 Програмні інтерфейси

Система повинна мати REST API для взаємодії з зовнішніми сервісами.

6. Вимоги до даних

- Дані повинні зберігатися у централізованій базі даних.
- Повинно бути реалізовано резервне копіювання не рідше одного разу на добу.

7. Вимоги до тестування та верифікації

- Кожна функціональна вимога повинна мати щонайменше один тестовий сценарій.
- Нефункціональні вимоги підлягають системному та валідаційному тестуванню.
- Повинна бути забезпечена трасованість «вимога – тест – результат».

8. Критерії приймання

Система вважається прийнятою, якщо:

- виконані всі функціональні вимоги;
- нефункціональні вимоги задоволені;
- критичних дефектів не виявлено.

У тестуванні:

SRS → основа для системного тестування
FR → test cases
NFR → quality attributes (ISO/IEC 25010)

A.2.2 Тестові дані для приймального тестування

Таблиця 39 - Значення параметрів ПЗ для приймального тестування

№	Параметр	Опис	Тип значення	Приклад
1	T_set	Задана температура (°C)	Вхідний параметр	180
2	T_init	Початкова температура (°C)	Вхідний параметр	25
3	P_max	Максимальна потужність нагрівача (Вт)	Константа	1000

			моделі	
4	Kp	Пропорційний коефіцієнт ПІД-регулятора	Налаштування	2.0
5	Ki	Інтегральний коефіцієнт	Налаштування	0.8
6	Kd	Диференційний коефіцієнт	Налаштування	0.1
7	Δt	Крок моделювання (с)	Параметр симуляції	0.1
8	Noise	Випадковий шум датчика (°C)	Вхідне збурення	±1.5
9	T_limit	Максимальна допустима температура (°C)	Обмеження безпеки	250

Таблиця 40 - *Опис тестових сценаріїв приймального тестування ПЗ*

D	Назва сценарію	Мета	Вхідні дані	Очікуваний результат	Фактичн. рез-т	Статус
S-01	Початковий нагрів	Перевірити, чи ПЗ дозволяє нагріватися до заданої температури	T_set=180°C, T_init=25°C, P_max=1000Вт	Досягнення стабільної температури в межах 178–182°C за ≤60с	—	Очікує виконання
S-02	Перегрів	Перевірити, чи ПЗ відключає підігрівач при T>250°C	T_set=240°C, Noise=+20°C	Потужність знижується до 0Вт при перевищенні 250°C	—	Очікує виконання
S-03	Реакція на різке охолодження	Імітація падіння температури на 50°C	ΔT=-50°C протягом 2с	Регулятор збільшує потужність >80% від P_max, потім стабілізує температуру	—	Очікує виконання
S-04	Стійкість до шуму сенсора	Перевірити стійкість ПІД-регулятора до випадкових коливань датчика	Noise=±1.5°C	Температура не виходить за межі ±2°C від T_set	—	Очікує виконання
S-05	Параметричний аналіз	Перевірити вплив зміни Kp, Ki, Kd	Kp=1.5..3.0, Ki=0.5..1.2, Kd=0.05..0.2	Визначено область стійких параметрів (без перерегулювання >10%)	—	Очікує виконання

Приклад переходу від сценарію до тесту

Сценарій TS-01: Початковий нагрів

Мета: перевірити, чи ПЗ дозволяє нагріватися до заданої температури 180°C за ≤60 с.

Таблиця 41-Тест TC-01.01 приймального тестування

Поле	Опис

ID тесту:	TC-01.01
Сценарій:	TS-01 – Початковий нагрів
Мета:	Перевірити стабілізацію температури
Передумови:	T_init = 25°C, P_max = 1000Вт, PID активний
Кроки:	1. Запустити симуляцію 2. Задати T_set = 180°C 3. Записати графік T(t) протягом 60 с
Очікуваний результат:	Температура досягає 180±2°C за 50–60 с
Фактичний результат	--
Статус:	Очікує виконання

А.2.3 Приклад висновку до підрозділу «Планування приймального тестування»

Планування приймального тестування забезпечило системний підхід до перевірки відповідності ПЗ вимогам замовника та специфікації. Було визначено основні тестові сценарії, вхідні дані та критерії успішності виконання тестів. Реалізація плану дозволить виявити дефекти до передачі ПЗ користувачу та підтвердити готовність ПЗ до експлуатації. *Таким чином, планування приймального тестування дозволило визначити основні критерії прийняття ПЗ, сформувати набір тестових сценаріїв і забезпечити готовність до проведення фінальної перевірки якості ПЗ*

А.3 Планування системного тестування

Приклад тестових даних для системного тестування включає тестові сценарії, дані та очікувані результати

Таблиця 42 - Тестові сценарії системного тестування

№	Тестовий сценарій	Вхідні дані	Очікуваний результат	Тип тестування	Примітки
1	Авторизація користувача	Логін: user, Пароль: 1234	Успішний вхід у ПЗ	Функціональне	Перевірка базової функції входу
2	Невдала авторизація	Логін: user, Пароль: wrong	Відмова в доступі, повідомлення про помилку	Функціональне	Перевірка оброблення некоректних даних
3	Створення нового запису	Дані: {ім’я, прізвище, email}	Запис успішно додано до БД	Функціональне	Перевірка CRUD-операцій
4	Видалення запису	ID запису = 5	Запис видалено, відображення актуальної таблиці	Функціональне	Перевірка інтеграції з базою даних
5	Оброблення некоректних	Дані: {ім’я = ""}	Показ повідомлення	Функціональне	Валідація полів форми

	даних		про помилку, запис не додано		
6	Генерація звіту	Параметри: дата = 01.01.2025	Звіт створено у форматі PDF	Нефункціональне	Перевірка звітності
7	Відповідь ПЗ під навантаженням	1000 одночасних запитів	Всі запити оброблено без помилок	Стрес-тестування	Перевірка продуктивності
8	Безпека передачі даних	Передача даних по HTTPS	Дані передаються зашифрованими	Безпекове	Перевірка шифрування та захисту
9	Відновлення після помилки	Вимкнення сервера на 1 хв	ПЗ автоматично відновлює роботу	Нефункціональне	Тестування відмовостійкості

Пояснення:

Тестовий сценарій – що саме перевіряється.

Вхідні дані – які дані вводяться для тестування.

Очікуваний результат – що ПЗ повинно видати або зробити.

Тип тестування – функціональне, нефункціональне, безпекове тощо.

Примітки – додаткова інформація або умови виконання тесту.

Приклад оформлення системних тестових сценаріїв та тестів.

Тестовий сценарій 1: Авторизація користувача

Мета: Перевірити, що користувач може успішно авторизуватись у ПЗ.

Таблиця 43 Тести першого тестового сценарію

№	Test Case	Вхідні дані	Очікуваний результат	Примітки
1	Авторизація з правильним логіном і паролем	Логін: user1, Пароль: 12345	Вхід у ПЗ успішний, відкривається головна сторінка	Повинно працювати для всіх валідних користувачів
2	Авторизація з неправильним паролем	Логін: user1, Пароль: 54321	Повідомлення про помилку "Невірний пароль"	Повторна спроба можлива
3	Авторизація з незареєстрованим користувачем	Логін: userX, Пароль: 12345	Повідомлення "Користувач не знайдений"	Перевірка оброблення неіснуючих логінів
4	Порожні поля логін/пароль	Логін: "", Пароль: ""	Повідомлення "Будь ласка, введіть логін та пароль"	Перевірка валідації форми

Тестовий сценарій 2: Додавання нового елементу

Мета: Перевірити функцію створення нового запису у ПЗ.

Таблиця 44 Тести другого тестового сценарію

№	Test Case	Вхідні дані	Очікуваний результат	Примітки
1	Додавання коректного елементу	Назва: "Тестовий об’єкт", Тип: "Тип	Запис успішно збережений,	Перевірка базового

		А"	відображається у списку	сценарію
2	Додавання елементу з порожньою назвою	Назва: "", Тип: "Тип А"	Повідомлення "Назва не може бути порожньою"	Перевірка валідації полів
3	Додавання елементу з довгою назвою (>255 символів)	Назва: 300 символів, Тип: "Тип А"	Повідомлення про перевищення довжини	Перевірка обмежень бази даних
4	Додавання дубліката	Назва: "Тестовий об'єкт", Тип: "Тип А"	Повідомлення "Запис вже існує"	Перевірка унікальності

Тестовий сценарій 3: Вихід з ПЗ
Мета: Перевірити коректність завершення сеансу користувача.

Таблиця 45 Тести третього тестового сценарію

№	Test Case	Вхідні дані	Очікуваний результат	Примітки
1	Вихід після авторизації	Авторизований користувач	Сеанс завершено, користувач повертається на сторінку логіну	Перевірка коректного очищення сесії
2	Вихід без авторизації	Користувач не авторизований	Кнопка "Вихід" недоступна	Перевірка доступності функції

Примітки:
1. Тестові сценарії описують логічні функціональні блоки ПЗ.
2. Test Case – конкретна перевірка у межах сценарію, з конкретними даними та очікуваним результатом.
3. Для системного тестування слід включати як функціональні, так і *нефункціональні перевірки* (продуктивність, безпека, відмовостійкість).

Сценарій 9: Перевірка безпеки. Мета: Перевірка доступу до обмежених функцій.

Таблиця 46 Тести Перевірки безпеки

№	Test Case	Вхідні дані	Очікуваний результат
1	Доступ адміна	Логін: admin	Повний доступ до функцій
2	Доступ користувача	Логін: user	Обмежений доступ, немає редагування
3	Доступ без логіну	—	Перенаправлення на авторизацію
4	Спроба SQL-ін'єкції	Логін: ' OR 1=1; —	Вхід заборонено, ПЗ не зламане

Сценарій 10: Продуктивність. Мета: Перевірка роботи ПЗ при високому навантаженні.

Таблиця 47 Тести Продуктивності

№	Test Case	Вхідні дані	Очікуваний результат

1	100 одночасних користувачів	Симуляція навантаження	ПЗ стабільне, затримка <2с
2	10^3 одночасних користувачів	Симуляція навантаження	ПЗ працює, час відповіді <5с
3	Масове додавання записів	1000 записів одночасно	Записи збережені, без помилок
4	Масове видалення	1000 записів одночасно	Оброблення без зависань

Приклад висновку про планування системного тестування

Завдяки ретельному плануванні системного тестування визначені обсяг тестування, що охоплює всі ключові функціональні модулі та критичні бізнес-процеси ПЗ, а також важливі нефункціональні аспекти: продуктивність, безпека та стійкість до збоїв. Розроблені основні тестові сценарії, які передбачають перевірку нормальної роботи ПЗ, граничних станів та можливих ситуацій відмов.

Очікувані результати тестування включають підтвердження коректності роботи всіх функціональних блоків, відповідність показників продуктивності технічним вимогам, а також виявлення та документування дефектів, що можуть впливати на роботу ПЗ. Критерії успішності тестів визначено як повну відповідність функціональності та вимогам без критичних помилок.

Оцінка готовності до виконання системного тестування показала, що тестові дані, середовище та інструменти підготовлені для виконання тестів. За результатами планування рекомендовано приступати до розроблення інтеграційного тестування.

А.4 Планування інтеграційного тестування

План інтеграційного тестування потрібен мати таку структуру

1. Вступ

Мета документа: Описати підхід, обсяг, ресурси, графік і критерії інтеграційного тестування компонентів ПЗ.

Об’єкт тестування: ПЗ керування орієнтацією супутника.

2. Цілі інтеграційного тестування:

Перевірити правильність взаємодії між модулями.

Виявити дефекти, що виникають під час обміну даними.

Переконатися, що компоненти ПЗ працюють як єдине ціле.

Підтвердити відповідність інтегрованого ПЗ вимогам до інтерфейсів.

3. Обсяг тестування

До інтеграції включаються такі модулі:

Модуль датчиків кутової швидкості.

Модуль фільтрації сигналів.

Модуль обчислення орієнтації.

Модуль керування моментними двигунами.

4. Стратегія інтеграції

Тип підходу: змішаний (bottom-up + top-down). Порядок інтеграції:

Інтеграція сенсорного та фільтраційного модулів.

Інтеграція блоку орієнтації з фільтром.

Додавання контролера моментних двигунів.

Інструменти: GoogleTest, CTest тощо.

5. Критерії початку інтеграційного тестування:

Усі модулі пройшли модульне тестування.

Доступні специфікації інтерфейсів.

Підготовлене тестове середовище та заглушки.

Критерії завершення інтеграційного тестування:

Пройдено всі критичні сценарії без дефектів.

Виправлені всі знайдені помилки категорії «високий пріоритет».

Отримано позитивний висновок про стабільність взаємодії.

6. Тестові дані

Таблиця 48 Тестові сценарії інтеграційного тестування

№	Назва сценарію	Опис	Очікуваний результат
1	Обмін між сенсором і фільтром	Передача даних із шумом до фільтра	Дані правильно фільтруються
2	Передача даних фільтра в блок орієнтації	Обчислення кута орієнтації	Кут орієнтації відповідає моделі
3	Передача керуючого сигналу двигунам	Вихід PWM у заданих межах	Сигнал стабільний, без перевищень
4	Синхронізація частот оновлення	Узгодження таймерів	Дані приходять без затримок >10 мс

Тестові параметри

Симульовані вектори кутових швидкостей (ωx, ωy, ωz).

Таблиці коефіцієнтів фільтра Калмана.

Початкові орієнтаційні кути (roll, pitch, yaw).

Значення моментів, обмеження по струму.

7. Ресурси та середовище

Обладнання: Linux-сервер, STM32 симулятор.

ПЗ: CMake, CPPUnit, Boost.Units.

Команда: 1 розробники, 1 тестувальник, 1 аналітик.

9. План-графік

Таблиця 49 План графік виконання інтеграції

Етап	Дата початку	Дата завершення	Відповідальний
Підготовка середовища	01.11.2025	03.11.2025	Інженер з тестування
Інтеграція сенсорних модулів	04.11.2025	06.11.2025	Розробник №1
Інтеграція блоку керування	07.11.2025	09.11.2025	Розробник №2
Виконання тестів	10.11.2025	13.11.2025	QA
Аналізування результатів	14.11.2025	15.11.2025	Аналітик

8. Ризики

Несумісність форматів даних між модулями.
Затримки у передачі сигналів.
Відсутність емуляторів апаратних пристроїв.

План зниження ризиків:

Використання інтерфейсів із жорсткою типізацією.
Верифікація протоколів обміну через SITL.

9. Критерії прийняття

Не більше 1 незначного дефекту в нефункціональних вимогах.
Всі функціональні тести успішно пройдені.
ПЗ працює стабільно ≥ 1 год у тестовому режимі.

10. Висновок

План інтеграційного тестування дозволяє послідовно перевірити коректність взаємодії між компонентами ПЗ, забезпечити стабільність та узгодженість функцій при інтеграції, а також знизити ризики помилок перед системним тестуванням.

Тестові дані для інтеграційного тестування – це набір вхідних значень і сценаріїв, які використовуються для перевірки взаємодії між модулями. Дані мають охоплювати як типові, так і граничні та помилкові випадки.

Таблиця 50 Приклад тестових даних для інтеграційного тестування:

№	Інтеграція модулів	Вхідні дані	Очікуваний результат	Примітка
1	Модуль оброблення замовлення ↔ Модуль бази даних	Новий запис замовлення: {ID=101, товар="Дрон", кількість=2}	Запис успішно збережено, повернено підтвердження	Типовий сценарій
2	Модуль оброблення замовлення ↔ Модуль бази даних	Запис із дублікатом ID=101	Повідомлення про помилку "Запис вже існує"	Граничний випадок
3	Модуль автентифікації ↔ Модуль користувачів	Логін користувача: user1, пароль: correct	Доступ надається	Типовий сценарій
4	Модуль автентифікації ↔ Модуль користувачів	Невірний пароль	Відмова в доступі, повідомлення про помилку	Помилковий випадок
5	Модуль контролю температури ↔ Модуль відображення	Значення сенсора 75°C	Відображення 75°C, сигнал аварії не активний	Типовий сценарій
6	Модуль контролю температури ↔ Модуль відображення	Значення сенсора 120°C	Відображення 120°C, сигнал аварії активний	Граничний випадок

Рекомендації щодо формування тестових даних:

1. *Типові дані* – для перевірки стандартної поведінки ПЗ.

2. *Граничні значення* – для перевірки реакції на екстремальні умови.
3. *Невірні/помилкові дані* – для перевірки оброблення помилок.
4. *Комбінації модулів* – для перевірки взаємодії кількох модулів.

Приклад повної таблиці на 30 тестових сценаріїв для інтеграційного тестування ПЗ подано нижче. Вона охоплює різні модулі, типові, граничні та помилкові випадки

Таблиця 51 Повний перелік тестових сценаріїв

№	Інтеграція модулів	Вхідні дані	Очікуваний результат	Примітка
1	Модуль оброблення замовлення (МОЗ) ↔ База даних (БД)	Новий запис замовлення: {ID=101, товар="Дрон", кількість=2}	Запис успішно збережено, повернено підтвердження	Типовий сценарій
2	МОЗ ↔ БД	Дублікат ID=101	Повідомлення про помилку "Запис вже існує"	Граничний випадок
3	Модуль автентифікації (МА) ↔ Модуль користувачів (МК)	Логін user1, пароль correct	Доступ надається	Типовий сценарій
4	МА ↔ МЛ	Невірний пароль	Відмова в доступі	Помилковий випадок
5	Модуль контролю температури (МКТ) ↔ Модуль відображення (МВ)	Сенсор 75°C	Відображення 75°C, сигнал аварії не активний	Типовий сценарій
6	МКТ ↔ МВ	Сенсор 120°C	Відображення 120°C, сигнал аварії активний	Граничний випадок
7	Модуль оплати (МО)↔ МОЗ	Сума = 1000 грн	Оплата успішна, замовлення підтверджено	Типовий сценарій
8	МО ↔ МОЗ	Некоректна карта	Відмова в оплаті, повідомлення про помилку	Помилковий випадок
9	Модуль реєстрації (МР) ↔ МА	Новий користувач user2	Користувач зареєстрований, доступ активний	Типовий сценарій
10	МР ↔ МА	Логін уже існує	Повідомлення про помилку	Граничний випадок
11	Модуль генерації звітів ↔ БД	Звіт за 01.10–15.10	Звіт сформовано	Типовий сценарій
12	Модуль генерації звітів ↔ БД	Період без даних	Повідомлення "Немає даних для звіту"	Помилковий випадок
13	Модуль керування пристроєм ↔ Модуль сенсорів (МС)	Команда: увімкнути вентилятор	Вентилятор увімкнено, сенсор показує	Типовий сценарій

№	Інтеграція модулів	Вхідні дані	Очікуваний результат	Примітка
			обертів	
14	Модуль керування пристроєм ↔ МС	Сенсор не відповідає	ПЗ повідомляє про помилку	Помилковий випадок
15	Модуль логування ↔ Усі модулі	Подія: замовлення успішне	Подія записана у лог	Типовий сценарій
16	Модуль сповіщень ↔ МОЗ	Нове замовлення	Надіслано сповіщення користувачу	Типовий сценарій
17	Модуль сповіщень ↔ МОЗ	Некоректний e-mail	Сповіщення не надіслано, помилка зареєстрована	Помилковий випадок
18	Модуль кошика ↔ МОЗ	Додавання товару	Товар додано, кількість оновлено	Типовий сценарій
19	Модуль кошика ↔ МОЗ	Додавання від'ємної кількості	Повідомлення про помилку	Помилковий випадок
20	Модуль керування користувачами (МКК)↔ БД	Оновлення даних користувача	Дані успішно оновлені	Типовий сценарій
21	МКК ↔ БД	Неправильний формат даних	Повідомлення про помилку	Помилковий випадок
22	Модуль пошуку ↔ БД	Пошук існуючого товару	Показано товар	Типовий сценарій
23	Модуль пошуку ↔ БД	Пошук неіснуючого товару	Порожній результат, повідомлення	Граничний випадок
24	Модуль статистики ↔ БД	Запит по продажах	Дані сформовано	Типовий сценарій
25	Модуль статистики ↔ БД	Запит з некоректними параметрами	Повідомлення про помилку	Помилковий випадок
26	Модуль безпеки ↔ МА	Вхід з неавторизованого IP	Доступ заборонено	Типовий сценарій
27	Модуль безпеки ↔ МА	Атака Brute Force	ПЗ блокує IP	Помилковий сценарій
28	Модуль резервного копіювання ↔ БД	Планове резервне копіювання	Дані збережено	Типовий сценарій
29	Модуль резервного копіювання ↔ БД	Відсутність диску	Помилка резервного копіювання, повідомлення	Помилковий випадок
30	Модуль API ↔ Інші модулі	Запит на отримання замовлень	Коректна відповідь	Типовий сценарій

Приклад висновку до планування інтеграційного тестування

Планування інтеграційного тестування дозволило визначити послідовність і методику об'єднання окремих модулів у цілісне ПЗ, сфрормувати набір тестових сценаріїв, що перевіряють коректність

взаємодії між компонентами, обмін даними через інтерфейси та узгодженість функціональних блоків. Розроблений план забезпечив можливість поетапного виявлення помилок інтеграції та зменшення ризиків збоїв під час подальшого системного тестування та дозволив перейти до планування модульного тестування.

А.5 Планування модульного тестування

А.5.1 Приклад "Плану модульного тестування"

Модуль: SensorFilter
ПЗ**:** Орієнтаційне ПЗ CubeSat
Тип тестування: Модульне (Unit Testing)
Інструменти: CPPUnit
Версія: 1.0
Дата: 24.10.2025

1. Вступ

Мета документа: Описати підхід, середовище, сценарії та критерії проведення модульного тестування класу SensorFilter, який виконує цифрову фільтрацію даних з акселерометрів та гіроскопів CubeSat.
Об'єкт тестування**:** Клас SensorFilter, що реалізує середнє ковзне та фільтрацію низьких частот.
Область тестування: Перевіряється коректність математичних обчислень, оброблення шуму, граничних значень і стабільність алгоритму при постійних або насичених вхідних даних.
Посилання:

SRS документація
Design Document
Код: /src/sensors/SensorFilter.hpp, /src/sensors/SensorFilter.cpp

2. Об'єкти тестування

SensorFilter::addSample(double value)
SensorFilter::getFilteredValue()
SensorFilter::reset()
SensorFilter параметри (window_size, cutoff_frequency)

3. Середовище тестування

ОС: Linux Mint 22
Компілятор: g++ 13.2
Інструменти: GoogleTest 1.15, CMake 3.30
Бібліотеки: Boost.Units
Конфігурація: Debug, покриття з gcov/lcov

4. Стратегія тестування

Тип доступу: White-box (перевірка внутрішніх розрахунків середніх і коефіцієнтів).
Методи тестування:

Тестування межових значень

Перевірка виняткових ситуацій (ділення на нуль, пусте вікно)
Порівняння з еталонним результатом MATLAB/SciLab
Інструменти: mock-джерело даних (MockSensorSource)

5. Тестові сценарії та дані

Таблиця 52 Тестовий сценарій модульного тестування

№	Опис тесту	Вхідні дані	Очікуваний результат	Статус
1	Фільтрація сталої послідовності	100×[1.0]	1.0 ± 0.001	Очікує викон.
2	Середнє ковзне з шумом	[1, 1.1, 0.9, 1.2, 0.8]	1.0 ± 0.05	Очікує викон.
3	Оброблення насичених значень	[10, 10, 10, 10]	10.0	Очікує викон.
4	Перевірка reset()	після скидання	0.0	Очікує викон.
5	Фільтрація гармонічного сигналу	sin(ωt), ω = 1 рад/с	зменшення амплітуди у 2×	Очікує виконання

6. Критерії початку і завершення модульного тестування

Початок:

Код компілюється без помилок
Усі залежності бібліотек встановлено

Завершення:

Усі тести виконані
Не більше 5% неуспішних тестів низького пріоритету
Покриття коду ≥ 85%

7. План виконання

Підготовка середовища — 1 день
Розроблення тестів — 2 дні
Запуск тестів і збір результатів — 1 день
Аналіз покриття — 1 день
Виконавець: Студент гр. 649П — *Іваненко І.*

8. Критерії оцінювання результатів

Успішність виконання тестів
Відсутність виявлених помилок високої критичності
Рівень покриття функцій і умов

9. Ризики

Некоректна ініціалізація буфера при малому розмірі вікна
Нестабільні результати при коротких вибірках
Помилки округлення при double-арифметиці

10. Висновок

Очікується, що тестування підтвердить стабільність роботи модуля SensorFilter в умовах шуму та насичення, а також узгодження результатів з аналітичними обчисленнями. У разі успішного проходження тестів — модуль готовий до інтеграції у ПЗ орієнтації CubeSat.

А.5.2 Тестові дані для модульного тестування

Модуль: PIDController

ПЗ: Температурний регулятор промислового процесу

Мета: Перевірити правильність розрахунку вихідного сигналу регулятора залежно від помилки та параметрів (Kp, Ki, Kd).

Таблиця 53 Тестові дані модульного тестування

Параметр	Позн.	Тип	Значення за замовч.	Опис
Пропорційний коефіцієнт	Kp	double	2.0	Посилення пропорційної частини
Інтегральний коефіцієнт	Ki	double	0.5	Посилення інтегральної частини
Диференціальний коефіцієнт	Kd	double	0.1	Посилення похідної частини
Період дискретизації	Ts	double	0.1 с	Інтервал між вимірюван.
Задана температура	Tset	double	100 °C	Встановлене значення температури
Поточна температура	Tmeas	double	—	Змінна тестова величина
Вихідний сигнал	u(t)	double	—	Керуючий вплив на нагрівач

Tmeas — це значення температури, яке зчитується **з датчика** (сенсора).

Таблиця 54 Тестові сценарії для модульного тестування

№	Сценарій	Вхідні дані	Очікуваний результат	Примітка
1	Відсутність похибки	Tset=100; Tmeas=100	u=0	Регулятор стабільний
2	Початковий холодний об’єкт	Tset=100; Tmeas=20	u>0 (макс. потужність)	Пропорційна частина активна
3	Перегрів	Tset=100; Tmeas=110	u<0 (вимикає нагрів)	Перевірка негативного впливу
4	Мала похибка (стаціонарний режим)	Tset=100; Tmeas=99.8	u≈0.1	Вплив інтегральної складової
5	Різке зниження температури	Tset=100; Tmeas=80	u збільшується миттєво	Тест похідної частини
6	Тест інтегратора (накопичення похибки)	Послідовно: [95, 96, 97, 98, 99]	u поступово зменшується	Перевірка антивіндапу
7	Випадковий шум сенсора	Tmeas=100±0.5	Вихід фільтрований, без коливань > ±0.2	Стійкість до шуму
8	Граничні значення виходу	Tset=200; Tmeas=0	u=Umax	Перевірка насичення
9	Негативне значення коефіцієнтів	Kp=-1.0	Помилка або виняток	Перевірка валідації

10	Нульовий період дискретизації	Ts=0	Виняток divide-by-zero	Тест захисту від помилок

Критерії прийнятності результатів
Відхилення між обчисленим і очікуваним виходом ≤ 1%.
Усі тести завершуються без аварійного завершення програми.
Для шумових сценаріїв коливання виходу не перевищують 5%.

А.5.3 Приклад висновку про планування модульного тестування

У процесі планування модульного тестування визначили мету, об'єкти, обсяг, стратегію та ресурси, необхідні для проведення перевірки окремих компонентів розробленого програмного забезпечення. Основна мета — забезпечення перевірки правильності роботи кожного модуля ПЗ в ізоляції, до початку інтеграційного тестування.

Визначено об'єкти тестування — класи, функції та модулі, що реалізують основну функціональність ПЗ (наприклад, модулі керування, обчислень, введення/виведення).

Сформовано критерії проходження тестів для кожного модуля, включно з діапазонами допустимих значень, обмеженнями на час виконання та перевіркою коректності типів і розмірностей даних.

Обрано середовище тестування — MS Visual Studio Community.

Визначено стратегію тестування — застосовано метод *white-box testing* із фокусом на функціональну правильність і граничні умови.

Сформовано набір тестових даних**,** що охоплює типові, граничні та некоректні випадки, включно зі сценаріями для перевірки оброблення виключних ситуацій.

План модульного тестування, який визначає порядок, критерії та ресурси виконання тестування, що забезпечило системність і відтворюваність перевірок та є основою для подальшого інтеграційного тестування.

Таким чином:

1. Визначено обсяг і межі модульного тестування.
2. Встановлено засоби, ресурси та критерії оцінки результатів.
3. Підготовлено повний набір тестових сценаріїв і тестових даних.
4. Забезпечено готовність до модульного тестування**.**

А.6 Виконання модульного тестування

А.6.1 Тести для перевірки ключових функцій ПЗ

Таблиця 55 Приклади модульних тестів

№	Назва тесту	Модуль	Вхідні дані	Очікуваний результат	Фактичний результат	Статус
1	SensorRead_ValidInput	SensorModule	Температура 25°C	Повертається 25 ±0.1°C	25.0°C	Pass
2	PID_ZeroError	PIDController	Error = 0°C	Output = 0	0	Pass
3	PID_OverTempResp	PIDController	Error =	Output	Output ↓	Pass

	onse		+20°C	зменшується		
4	Heater_ZeroPower	HeaterControl	PowerCmd = 0	PWM = 0	0	Pass
5	Heater_MaxPower	HeaterControl	PowerCmd = 100%	PWM = 255	255	Pass

А.6.2 Приклад висновку про модульне тестування

Метою модульного тестування є перевірка коректності реалізації окремих компонентів ПЗ керування орієнтацією супутника, зокрема:

класів PIDController, GyroSensor, ReactionWheel, OrientationEstimator;

правильності обчислення фізичних величин у межах бібліотеки (Boost.Units);

перевірки стабільності обчислень при малих та великих значеннях вхідних сигналів.

Таблиця 56 Об'єкти модульного тестування ПЗ:

№	Модуль	Призначення	Тестовий файл
1	PIDController	Формує керуючий момент за похибкою орієнтації	test_pid_controller.cpp
2	GyroSensor	Імітує вимірювання кутової швидкості з шумом	test_gyro_sensor.cpp
3	ReactionWheel	Реалізує динаміку виконавчого механізму	test_reaction_wheel.cpp
4	OrientationEstimator	Обчислює кватерніон орієнтації на основі вимірів	test_orientation_estimator.cpp

Методика тестування

Тестування виконувалось із застосуванням фреймворку Google Test. Методологія — *white-box testing* із фокусом на коректність формул і перевірку граничних умов.

Тестові дані генерувались як:

фіксовані сценарії (e(t) — сталої похибки);

псевдовипадкові зразки шуму (gyro_noise ~ N(0, σ^2)).

Для кожного тесту було визначено очікуваний результат і критерії проходження:

```
EXPECT_NEAR(pid.output(), expected_value, tolerance);
EXPECT_TRUE(isfinite(sensor.reading()));
EXPECT_LE(abs(moment - reference_moment), 1e-6);
```

Таблиця 57 Результати тестування

№	Модуль	Кількість тестів	Пройдено	Не пройдено	Примітки
1	PIDController	12	12	0	стабільна робота при Kp, Ki, Kd > 0
2	GyroSensor	8	8	0	відхилення не перевищує σ = 0.01 rad/s

3	ReactionWheel	10	9	1	незначна похибка при насиченні моменту
4	OrientationEstimator	9	9	0	точність 10^{-6} при dt = 0.01 s

Аналіз результатів

Всі основні модулі функціонують відповідно до очікуваної специфікації.
Виявлена незначна розбіжність при тестуванні насичення моменту у ReactionWheel, рекомендовано уточнити обмеження насичення.
Фізичні одиниці всіх розрахункових змінних відповідають SI.
Середній час виконання одного тесту — 0.8 мс, що відповідає вимогам реального часу.

Висновки

1. Модулі PIDController, GyroSensor, OrientationEstimator пройшли всі заплановані тести без помилок.
2. Всі основні модулі працюють відповідно до специфікації вимог.
3. Модуль ReactionWheel потребує незначного уточнення в моделі насичення.

Рівень покриття коду тестами — **92%,** що є прийнятним для даного етапу розроблення.

4. Модульне тестування підтвердило відповідність компонентів функціональним вимогам та їхню готовність до етапу інтеграційного тестування**.**

Під час тестування було виявлено та усунуто кілька незначних помилок у реалізації логіки ПІД-регулятора, після чого повторне тестування підтвердило правильність роботи.
У результаті модульного тестування підтверджено правильність алгоритмів вимірювання, регулювання та керування потужністю.
Отримані результати свідчать про стабільну роботу компонентів, що дозволяє перейти до етапу інтеграційного тестування ПЗ.

А.7 Виконання інтеграційного тестування

Нижче наведено розширений звіт про виконання інтеграційного тестування для CubeSat Orientation Control System, де в кожному тестовому сценарії вказано конкретні комбінації модулів (збірки) та заглушки (stubs / mocks), які використовувалися для імітації ще не реалізованих або зовнішніх компонентів.

Система: Орієнтаційна система CubeSat (3-осьова стабілізація)

1 Мета: Перевірка взаємодії між модулями орієнтаційної системи CubeSat після їх інтеграції. Тестування спрямовано на виявлення помилок інтерфейсів між компонентами та підтвердження коректного оброблення.

2. Об’єкт тестування

Таблиця 58 Склад інтегрованої збірки

Модуль	Призначення	Реалізація
Sensor	Збір даних із гіроскопа, магнітометра, сонячного датчика	реалізовано
FilterEKF	Фільтр Калмана для оцінки орієнтації	реалізовано
Attituder	PID-регулятор орієнтації	реалізовано
Actuator	Реакційні колеса та магнітні котушки	заглушка (Stub)
Interface	Обмін з бортовим комп’ютером	заглушка (Mock)

3. Середовище тестування

Платформа: Linux Mint 22, GCC 13.2, Boost.Test

Симуляційне середовище: SITL (Software-In-The-Loop)

Тестові інтерфейси: C++ з Boost.Units (SI-бібліотека)

Частота циклу: 100 Гц

Модель: CubeSat 3U (3 реакційні колеса, 3 магнітометри, 1 сонячний датчик)

4. Тестові сценарії

Таблиця 59 Тестові сценарії (із вказаними збірками та заглушками)

№	Назва сценарію	Склад збірки	Викор. Заглуш-ки	Опис тесту	Очік. рез.	Факт. Рез.	Ста-тус
1	Перевірка обміну даними між сенсорами і фільтром Калмана	Sensor + FilterEKF	*Немає*	Сенсор генерує ωx, ωy, ωz; перевіряється надходження даних у фільтр	Дані зчитано і нормалізовано	Дані коректно прийняті	Pass
2	Тестування оцінки орієнтації фільтром	Sensor + FilterEKF + Interface	Interface = Mock	Симуляція шумних даних, передача результату через інтерфейс	Кватерніон q нормалізований	q=(0.707,0.707,0,0)	Pass
3	Інтеграція фільтра та PID-контролера	FilterEKF + Attituder	*Actuator = Stub*	Вхід: qref=(1,0,0,0), qmeas=(0.99,0.01,0,0)	Розрахунок моменту M	M=(0.001,0.003,0.000) Н·м	Pass
4	Перевірка взаємодії контролера та актуатора	Attituder + Actuator	*Actuator = Stub з логом команд*	Перевірка передачі моменту на колеса	Отримано команду на момент M	M передано у log.txt	Pass
5	Перевірка переходу у safe-mode	Sensore +	*Sensor= Fault Injection*	Імітація відмови одного сенсора	Перехід у безпечний режим	safe-mode=true	Pass

		Filter EKF + Attitude	*Stub*				

5. Реалізація заглушок

Actuator Stub

```
class ActuatorStub {
public:
    void applyTorque(Vector3 torque) {
        std::ofstream log("log.txt", std::ios::app);
        log << "Apply torque: " << torque.x << ", "
            << torque.y << ", " << torque.z << "\n";
    }
};
```

CommInterface Mock

```
class CommInterfaceMock {
public:
    void sendQuaternion(const Quaternion& q) {
        std::cout << "[Mock] Sending quaternion: " << q << std::endl;
        lastSent = q;
    }
    Quaternion lastSent;
};
```

Sensor Fault Stub

```
class SensorFaultStub : public SensorModule {
public:
    bool sensorFail = false;
    Vector3 readGyro() override {
        if(sensorFail) throw std::runtime_error("Gyro failure");
        return {0.05, 0.03, 0.02};
    }
};
```

6. Результати виконання

5 сценаріїв виконано; усі успішно пройшли перевірку.

Латентність передачі між модулями ≤ 5 мс.

При відмові сенсора — система переходить у safe-mode за 0.3 с.

Вихідні кватерніони нормалізовані ($\|q\| \approx 1$).

PID-вихід стабільний без коливань.

7. Висновки

Інтеграційне тестування продемонструвало коректну взаємодію між усіма функціональними модулями CubeSat. Використання заглушок дозволило протестувати обмін сигналами, навіть без фізичного актуатора та бортового комп’ютера. ПЗ продемонструвало відповідність функціональним вимогам щодо стабілізації, реакції на відмову та часових обмежень.

8. Рекомендації

Замінити заглушки на апаратні драйвери під час подальшого тестування. Додати стрес-тести на перевантаження комунікаційного каналу. Розширити внесення відмов (fault-injection) для різних типів сенсорів.

А.8 Виконання системного тестування

Приклад виконання системних тестів

Таблиця 60 Сценарій 1: Авторизація користувача

№	Test Case	Вхідні дані	Очікуваний результат	Фактичний результат	Статус
1	Успішна авторизація	Логін: user1, Пароль: 12345	Вхід успішний, головна сторінка відкривається	Вхід успішний, головна сторінка відкрилася	Pass
2	Невірний пароль	Логін: user1, Пароль: 54321	Повідомлення "Невірний пароль"	Повідомлення відображено	Pass
3	Незареєстрований користувач	Логін: userX, Пароль: 12345	Повідомлення "Користувач не знайдений"	Повідомлення відображено	Pass
4	Порожні поля	Логін: "", Пароль: ""	Повідомлення "Будь ласка, введіть логін та пароль"	Повідомлення відображено	Pass

Таблиця 61 Сценарій 2: Додавання нового об'єкту

№	Test Case	Вхідні дані	Очікуваний результат	Фактичний результат	Статус
1	Коректні дані	Назва: "Тест", Тип: "А"	Запис додано успішно	Запис додано успішно	Pass
2	Порожня назва	Назва: "", Тип: "А"	Повідомлення "Назва не може бути порожньою"	Повідомлення відображено	Pass
3	Занадто довга назва	Назва: 300 символів, Тип: "А"	Повідомлення про перевищення довжини	Повідомлення відображено	Pass
4	Дублікат запису	Назва: "Тест", Тип: "А"	Повідомлення "Запис вже існує"	Повідомлення відображено	Pass

Таблиця 62 Сценарій 3: Вихід із системи

№	Test Case	Вхідні дані	Очікуваний результат	Фактичний результат	Статус
1	Вихід після входу	Авторизований користувач	Сеанс завершено, повернення на	Вихід виконано, повернення на логін	Pass

			логін		
2	Вихід без авторизації	Неавторизований користувач	Кнопка "Вихід" недоступна	Кнопка недоступна	Pass
3	Вихід із неактивної сесії	Просрочена сесія	Автоматичний редірект на логін	Перенаправлення на логін	Pass
4	Вихід при відкритій формі	Заповнена форма	Попередження "Зберегти зміни?"	Повідомлення відображено	Pass

Таблиця 63 Сценарій 4: Редагування об'єкту

№	Test Case	Вхідні дані	Очікуваний результат	Фактичний результат	Статус
1	Коректне редагування	Змінити назву на "Новий"	Запис оновлено	Запис оновлено	Pass
2	Порожня назва	Назва: ""	Повідомлення про помилку	Повідомлення відображено	Pass
3	Некоректний формат	Назва: "@!#"	Повідомлення "Невірний формат"	Повідомлення відображено	Pass
4	Редагування неіснуючого об'єкту	ID: 9999	Повідомлення "Об'єкт не знайдено"	Повідомлення відображено	Pass

Приклад висновку по виконанню системних тестів

1. Всі функції ПЗ протестовані на відповідність вимогам SRS.
2. Усі критичні сценарії пройшли успішно.
3. Виявлено, що *не було серйозних помилок,* дрібні UI-помилки (не критичні) відсутні або будуть виправлені.
4. ПЗ готове до використання у тестовому середовищі або до приймального тестування.

А.9 Виконання приймального тестування

Приклад виконання приймального тестрування

Таблиця 64 Приклад виконання тесту

Поле	Опис
ID тесту	TC-01.01
Сценарій:	TS-01 – Початковий нагрів
Мета:	Перевірити стабілізацію температури
Передумови:	Tinit = 25°C, Pmax = 1000Вт, PID активний
Кроки:	1. Запустити симуляцію 2. Задати T_set = 180°C 3. Записати графік T(t) протягом 60 с
Очікуваний результат:	Температура досягає 180±2°C за 50–60 с
Фактичний результат	179.6°C за 54 с
Статус:	Pass

А.10 Приклади загальних висновків до курсового проєкту

У процесі тестування ПЗ були проведені перевірки, що охоплюють усі основні рівні контролю якості: модульне**,** інтеграційне**,** системне **та** приймальне тестування**.** Кожен рівень тестування мав свою мету, критерії та результати, що забезпечили поступове підвищення рівня впевненості у правильності роботи ПЗ.

Модульне тестування дозволило перевірити окремі програмні модул та виявити локальні помилки логіки, некоректне оброблення граничних значень і неточності в алгоритмах. Проведені тести підтвердили коректність базових функцій та оброблення виключних ситуацій.

Інтеграційне тестування здійснило поступове об'єднання модулів і перевірку їх взаємодії через визначені інтерфейси. Це дало змогу переконатися у правильності обміну даними, сумісності типів та стабільності ПЗ при складних взаємозв'язках компонентів. Виявлені неузгодженості було усунуто до початку наступного етапу.

Системне тестування забезпечило перевірку роботи всього ПЗ в цілому, включно з перевіркою функціональних, нефункціональних та експлуатаційних вимог. Особлива увага приділялась продуктивності, стійкості до збоїв і поведінці ПЗ в граничних режимах. Отримані результати підтвердили відповідність ПЗ встановленим технічним вимогам і критеріям якості.

Приймальне тестування підтвердило відповідність ПЗ вимогам замовника та готовність до впровадження. Всі ключові функції виконуються відповідно до специфікації вимог, а виявлені незначні відхилення не впливають на загальну працездатність і надійність ПЗ.

Таким чином, проведені випробування підтвердили, що ПЗ відповідає встановленим вимогам, має високу надійність, і може бути рекомендоване для використання.

Висновок 2

У межах курсового проєкту з дисципліни **«Тестування та верифікація програмного забезпечення»** було виконано комплекс робіт, спрямованих на забезпечення якості ПЗ, розробленого в межах кваліфікаційної роботи бакалавра. Основною метою проєкту було набуття практичних навичок застосування методів і засобів тестування та верифікації ПЗ, а також закріплення теоретичних знань, отриманих під час вивчення дисципліни.

У процесі виконання курсового проєкту було проаналізовано вимоги до ПЗ, сформовано їх формалізований перелік та встановлено відповідність вимог характеристикам якості згідно з сучасною моделлю **ISO/IEC 25010**. На основі цього було розроблено повний комплект тестової документації,

що включає плани модульного, інтеграційного, системного та валідаційного (приймального) тестування.

Відповідно до розроблених планів було виконано всі передбачені види тестування ПЗ, задокументовано результати тестування, виявлено та проаналізовано дефекти, а також оцінено ступінь відповідності ПЗ функціональним і нефункціональним вимогам. За результатами тестування встановлено, що критичних дефектів не виявлено, а виявлені некритичні зауваження не впливають на коректність та стабільність роботи системи.

Отримані результати тестування свідчать про те, що ПЗ **відповідає визначеним вимогам, має належний рівень якості та є готовим до використання за призначенням**. Система може бути рекомендована до впровадження та подальшої експлуатації.

У ході виконання курсового проєкту здобувач набув практичного досвіду застосування сучасних інструментів тестування, засобів документування та аналізу результатів, а також сформував навички системного підходу до забезпечення якості. Отримані знання та вміння є важливими для подальшої професійної діяльності у сфері інформаційних технологій та можуть бути використані під час розроблення, тестування та супроводу ПЗ різного рівня складності.

Таким чином, поставлені завдання курсового проєкту були повністю виконані, а досягнуті результати підтверджують доцільність та ефективність застосування методів тестування та верифікації ПЗ як невід’ємної складової процесу його створення.

БІБЛІОГРАФІЧНИЙ СПИСОК

1 CppUnit Cookbook https://cppunit.sourceforge.net/doc/cvs/cppunit_cookbook.html Доступ 20.09.2025

2 Jest is a delightful JavaScript Testing Framework with a focus on simplicity https://jestjs.io/ Доступ 20.09.2025

3 Junit https://junit.org/ Доступ 20.09.2025

4 NUnit https://nunit.org/ Доступ 20.09.2025

5 Jest is a delightful JavaScript Testing Framework with a focus on simplicity https://cucumber.io/ Доступ 20.09.2025

6 SoapUI: The World's Most Popular API Testing Tool https://www.soapui.org/ Доступ 20.09.2025

7 Postman: The World's Leading API Platform | Sign Up for Free https://www.postman.com/ Доступ 20.09.2025

8 Appium documentation https://appium.io/docs/en/2.0/ Доступ 20.09.2025

9 Cypress: Testing Frameworks for Javascript | Write, Run, Debug https://www.cypress.io/ Доступ 20.09.2025

10 Playwright: Fast and reliable end-to-end testing for modern web apps https://playwright.dev/ Доступ 20.09.2025

11 Selenium https://www.selenium.dev/ Доступ 20.09.2025

12 The #1 AI-Native Test Automation Platform https://www.mabl.com/ https://www.mabl.com/ Доступ 20.09.2025

13 Automation-friendly framework for Continuous Testing https://gettaurus.org/kb/Index/ Доступ 20.09.2025

14 Locust - A modern load testing framework https://locust.io/ Доступ 20.09.2025

15 ab - Apache HTTP server benchmarking tool https://httpd.apache.org/docs/2.4/programs/ab.html Доступ 20.09.2025

16 The AI-Powered Testing Platform Built for Speed, Scalability and Accuracy https://applitools.com/ Доступ 20.09.2025

ЗМІСТ

ПЕРЕДМОВА 3
Сучасна модель якості ПЗ за стандартами ISO/IEC 5
Зв'язок характеристик якості ISO/IEC 25010 з типами тестів 10
Системи критичного призначення 23
Вибір типів тестування для критичних систем 24
Вступ до курсового проєкту 27
Практична 1 «Планування приймального тестування» 28
Практична робота 2 «Планування системного тестування » 34
Практична робота 3 «Планування інтеграційного тестування» 38
Практична робота 4 «Планування модульного тестування» 44
Практична робота 5 «Виконання модульного тестування» 48
Практична робота 6 «Виконання інтеграційного тестування» 51
Практична робота 7 «Виконання системного тестування» 54
Практична робота 8 «Виконання приймального тестування» 58
Висновок курсового проєкту 62
ПІСЛЯМОВА 63
ДОДАТОК А. Тестування ПЗ кваліфікаційної роботи бакалавра 64
А.1 Приклад вступу до третього розділу бакалаврської роботи 64
А.2 Планування приймального тестування 64
А.2.1 Приклад System Requirements Specification (SRS) 64
А.2.2 Тестові дані для приймального тестування 66
А.2.3 Приклад висновку до підрозділу «Планування приймального тестування» 68
А.3 Планування системного тестування 68
А.4 Планування інтеграційного тестування 71
А.5 Планування модульного тестування 76
А.5.1 Приклад "Плану модульного тестування" 76
А.5.2 Тестові дані для модульного тестування 78
А.5.3 Приклад висновку про планування модульного тестування 79
А.6 Виконання модульного тестування 79
А.6.1 Тести для перевірки ключових функцій ПЗ 79
А.6.2 Приклад висновку про модульне тестування 80
А.7 Виконання інтеграційного тестування 81
А.8 Виконання системного тестування 84
А.9 Виконання приймального тестування 85
А.10 Приклади загальних висновків до курсового проєкту 86
БІБЛІОГРАФІЧНИЙ СПИСОК 88

Навчальне видання

Клименко Тетяна Анатоліївна
Манжос Юрій Семенович

Тестування та верифікація програмного забезпечення (курсовий проєкт)

Редактор

Зв. план, 2026
Підписано до видання 00.00.2026
Ум. друк. арк. 0,0. Обл.-вид. арк. 0,00. Електронний ресурс

Видавець і виготовлювач
Національний аерокосмічний університет ім. М. Є. Жуковського
«Харківський авіаційний інститут»
61070, Харків-70, вул. Манька, 17
http://www.khai.edu
Видавничий центр «ХАІ»
61070, Харків-70, вул. Чкалова, 17
izdat@khai.edu

Свідоцтво про внесення суб’єкта видавничої справи
до Державного реєстру видавців, виготовлювачів і розповсюджувачів
видавничої продукції сер. ДК № 391 від 30.03.2001